

✓ Economics 430: Project 1

- Tanishq Tiwari
- Julia Miskiewicz
- Madison Richards

```
!pip install --upgrade pandas-profiling
import pandas as pd
```

```
Requirement already satisfied: pandas-profiling in /usr/local/lib/python3.10/dist-packages (3.6.6)
Requirement already satisfied: ydata-profiling in /usr/local/lib/python3.10/dist-packages (from pandas-profiling) (4.11.0)
Requirement already satisfied: scipy<1.14,>=1.4.1 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling->pandas-profiling) (1.13.1)
Requirement already satisfied: pandas!=1.4.0,<3,>1.1 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling->pandas-profiling) (2.2.2)
Requirement already satisfied: matplotlib<3.10,>=3.5 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling->pandas-profiling) (3.7.1)
Requirement already satisfied: pydantic>=2 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling->pandas-profiling) (2.9.2)
Requirement already satisfied: PyYAML<6.1,>=5.0.0 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling->pandas-profiling) (6.0.2)
Requirement already satisfied: Jinja2<3.2,>=2.11.1 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling->pandas-profiling) (3.1.4)
Requirement already satisfied: visions<0.7.7,>=0.7.5 in /usr/local/lib/python3.10/dist-packages (from visions[type_image_path]<0.7.7,>=0.7.5->pandas-profiling) (0.7.5)
Requirement already satisfied: numpy<2.2,>=1.16.0 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling->pandas-profiling) (1.26.4)
Requirement already satisfied: htmlmin==0.1.12 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling->pandas-profiling) (0.1.12)
Requirement already satisfied: phik<0.13,>=0.11.1 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling->pandas-profiling) (0.12.4)
Requirement already satisfied: requests<3,>=2.24.0 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling->pandas-profiling) (2.32.3)
Requirement already satisfied: tqdm<5,>=4.48.2 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling->pandas-profiling) (4.66.5)
Requirement already satisfied: seaborn<0.14,>=0.10.1 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling->pandas-profiling) (0.13.2)
Requirement already satisfied: multimethod<2,>=1.4 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling->pandas-profiling) (1.12)
Requirement already satisfied: statsmodels<1,>=0.13.2 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling->pandas-profiling) (0.14.1)
Requirement already satisfied: typeguard<5,>=3 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling->pandas-profiling) (4.3.0)
Requirement already satisfied: imagehash==4.3.1 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling->pandas-profiling) (4.3.1)
Requirement already satisfied: wordcloud==1.9.3 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling->pandas-profiling) (1.9.3)
Requirement already satisfied: dacite>=1.8 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling->pandas-profiling) (1.8.1)
Requirement already satisfied: numba<1,>=0.56.0 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling->pandas-profiling) (0.60.0)
Requirement already satisfied: PyWavelets in /usr/local/lib/python3.10/dist-packages (from imagehash==4.3.1->ydata-profiling->pandas-profiling) (1.6.0)
Requirement already satisfied: pillow in /usr/local/lib/python3.10/dist-packages (from imagehash==4.3.1->ydata-profiling->pandas-profiling) (10.4.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.10/dist-packages (from Jinja2<3.2,>=2.11.1->ydata-profiling->pandas-profiling) (2.1.5)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib<3.10,>=3.5->ydata-profiling->pandas-profiling) (1.1.1)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.10/dist-packages (from matplotlib<3.10,>=3.5->ydata-profiling->pandas-profiling) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib<3.10,>=3.5->ydata-profiling->pandas-profiling) (4.53.0)
Requirement already satisfied: kiwisolver>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib<3.10,>=3.5->ydata-profiling->pandas-profiling) (1.4.5)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib<3.10,>=3.5->ydata-profiling->pandas-profiling) (24.1)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib<3.10,>=3.5->ydata-profiling->pandas-profiling) (3.2.0)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.10/dist-packages (from matplotlib<3.10,>=3.5->ydata-profiling->pandas-profiling) (2.9.0)
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in /usr/local/lib/python3.10/dist-packages (from numba<1,>=0.56.0->ydata-profiling->pandas-profiling) (0.43.0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/dist-packages (from pandas!=1.4.0,<3,>1.1->ydata-profiling->pandas-profiling) (2024.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.10/dist-packages (from pandas!=1.4.0,<3,>1.1->ydata-profiling->pandas-profiling) (2024.2)
Requirement already satisfied: joblib>=0.14.1 in /usr/local/lib/python3.10/dist-packages (from phik<0.13,>=0.11.1->ydata-profiling->pandas-profiling) (1.4.2)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.10/dist-packages (from pydantic>=2->ydata-profiling->pandas-profiling) (0.7.0)
Requirement already satisfied: pydantic-core==2.23.4 in /usr/local/lib/python3.10/dist-packages (from pydantic>=2->ydata-profiling->pandas-profiling) (2.23.4)
Requirement already satisfied: typing-extensions>=4.6.1 in /usr/local/lib/python3.10/dist-packages (from pydantic>=2->ydata-profiling->pandas-profiling) (4.12.2)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests<3,>=2.24.0->ydata-profiling->pandas-profiling) (3.3.2)
```

```
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests<3,>=2.24.0->ydata-profiling->pandas-prof:
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages (from requests<3,>=2.24.0->ydata-profiling->panda:
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages (from requests<3,>=2.24.0->ydata-profiling->panda:
Requirement already satisfied: patsy>=0.5.6 in /usr/local/lib/python3.10/dist-packages (from statsmodels<1,>=0.13.2->ydata-profiling->pandas-p
Requirement already satisfied: attrs>=19.3.0 in /usr/local/lib/python3.10/dist-packages (from visions<0.7.7,>=0.7.5->visions[type_image_path]<1
Requirement already satisfied: networkx>=2.4 in /usr/local/lib/python3.10/dist-packages (from visions<0.7.7,>=0.7.5->visions[type_image_path]<1
Requirement already satisfied: six in /usr/local/lib/python3.10/dist-packages (from patsy>=0.5.6->statsmodels<1,>=0.13.2->ydata-profiling->pan
```

```
!pip install simple_colors
```

```
➦ Requirement already satisfied: simple_colors in /usr/local/lib/python3.10/dist-packages (0.1.5)
```

```
pip install shap
```

```
➦ Requirement already satisfied: shap in /usr/local/lib/python3.10/dist-packages (0.46.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.10/dist-packages (from shap) (1.26.4)
Requirement already satisfied: scipy in /usr/local/lib/python3.10/dist-packages (from shap) (1.13.1)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.10/dist-packages (from shap) (1.5.2)
Requirement already satisfied: pandas in /usr/local/lib/python3.10/dist-packages (from shap) (2.2.2)
Requirement already satisfied: tqdm>=4.27.0 in /usr/local/lib/python3.10/dist-packages (from shap) (4.66.5)
Requirement already satisfied: packaging>20.9 in /usr/local/lib/python3.10/dist-packages (from shap) (24.1)
Requirement already satisfied: slicer==0.0.8 in /usr/local/lib/python3.10/dist-packages (from shap) (0.0.8)
Requirement already satisfied: numba in /usr/local/lib/python3.10/dist-packages (from shap) (0.60.0)
Requirement already satisfied: cloudpickle in /usr/local/lib/python3.10/dist-packages (from shap) (3.1.0)
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in /usr/local/lib/python3.10/dist-packages (from numba->shap) (0.43.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.10/dist-packages (from pandas->shap) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/dist-packages (from pandas->shap) (2024.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.10/dist-packages (from pandas->shap) (2024.2)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn->shap) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn->shap) (3.5.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.10/dist-packages (from python-dateutil>=2.8.2->pandas->shap) (1.16.0)
```

```
pip install boruta
```

```
➦ Requirement already satisfied: boruta in /usr/local/lib/python3.10/dist-packages (0.4.3)
Requirement already satisfied: numpy>=1.10.4 in /usr/local/lib/python3.10/dist-packages (from boruta) (1.26.4)
Requirement already satisfied: scikit-learn>=0.17.1 in /usr/local/lib/python3.10/dist-packages (from boruta) (1.5.2)
Requirement already satisfied: scipy>=0.17.0 in /usr/local/lib/python3.10/dist-packages (from boruta) (1.13.1)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn>=0.17.1->boruta) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn>=0.17.1->boruta) (3.5.0)
```

```
pip install scikit-learn
```

```
➦ Requirement already satisfied: scikit-learn in /usr/local/lib/python3.10/dist-packages (1.5.2)
Requirement already satisfied: numpy>=1.19.5 in /usr/local/lib/python3.10/dist-packages (from scikit-learn) (1.26.4)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn) (1.13.1)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn) (3.5.0)
```

```
pip install mlxtend
```

```

Requirement already satisfied: mlxtend in /usr/local/lib/python3.10/dist-packages (0.23.1)
Requirement already satisfied: scipy>=1.2.1 in /usr/local/lib/python3.10/dist-packages (from mlxtend) (1.13.1)
Requirement already satisfied: numpy>=1.16.2 in /usr/local/lib/python3.10/dist-packages (from mlxtend) (1.26.4)
Requirement already satisfied: pandas>=0.24.2 in /usr/local/lib/python3.10/dist-packages (from mlxtend) (2.2.2)
Requirement already satisfied: scikit-learn>=1.0.2 in /usr/local/lib/python3.10/dist-packages (from mlxtend) (1.5.2)
Requirement already satisfied: matplotlib>=3.0.0 in /usr/local/lib/python3.10/dist-packages (from mlxtend) (3.7.1)
Requirement already satisfied: joblib>=0.13.2 in /usr/local/lib/python3.10/dist-packages (from mlxtend) (1.4.2)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib>=3.0.0->mlxtend) (1.3.0)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.10/dist-packages (from matplotlib>=3.0.0->mlxtend) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib>=3.0.0->mlxtend) (4.54.1)
Requirement already satisfied: kiwisolver>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib>=3.0.0->mlxtend) (1.4.7)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib>=3.0.0->mlxtend) (24.1)
Requirement already satisfied: pillow>=6.2.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib>=3.0.0->mlxtend) (10.4.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib>=3.0.0->mlxtend) (3.2.0)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.10/dist-packages (from matplotlib>=3.0.0->mlxtend) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/dist-packages (from pandas>=0.24.2->mlxtend) (2024.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.10/dist-packages (from pandas>=0.24.2->mlxtend) (2024.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn>=1.0.2->mlxtend) (3.5.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.10/dist-packages (from python-dateutil>=2.7->matplotlib>=3.0.0->mlxtend) (1.

```

```
!pip install rpy2
%load_ext rpy2.ipython
```

```

Requirement already satisfied: rpy2 in /usr/local/lib/python3.10/dist-packages (3.4.2)
Requirement already satisfied: cffi>=1.10.0 in /usr/local/lib/python3.10/dist-packages (from rpy2) (1.17.1)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.10/dist-packages (from rpy2) (3.1.4)
Requirement already satisfied: pytz in /usr/local/lib/python3.10/dist-packages (from rpy2) (2024.2)
Requirement already satisfied: tzlocal in /usr/local/lib/python3.10/dist-packages (from rpy2) (5.2)
Requirement already satisfied: pycparser in /usr/local/lib/python3.10/dist-packages (from cffi>=1.10.0->rpy2) (2.22)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.10/dist-packages (from Jinja2->rpy2) (3.0.2)
The rpy2.ipython extension is already loaded. To reload it, use:
    %reload_ext rpy2.ipython

```

```

# Imports
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from numpy import random; random.seed(10) # pre-setting seed
from scipy import stats
from scipy.stats import norm, chi2, f
# Numpy version matters for scipy

# For model fitting
import statsmodels.api as sm
import statsmodels.stats.api as sms
import statsmodels.formula.api as smf
from statsmodels.graphics import regressionplots

```

```

from statsmodels.stats.multicomp import MultiComparison
from statsmodels.robust.robust_linear_model import RLM
from statsmodels.iolib.summary2 import summary_col
from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.stats.diagnostic import linear_reset
from sklearn.utils import resample
from sklearn.model_selection import train_test_split

# Subset regressions & feature selection
from sklearn.linear_model import LinearRegression
from mlxtend.feature_selection import ExhaustiveFeatureSelector as EFS
from mlxtend.feature_selection import SequentialFeatureSelector as SFS

# Heteroskedasticity
from statsmodels.stats.diagnostic import het_breuschpagan, het_white, het_goldfeldquandt

# Missing data imputation
from sklearn.impute import KNNImputer
from statsmodels.imputation import mice

```

Introduction

For project one, we used the Wooldridge dataset “Openness” to examine whether countries with greater imports proportional to their respective GDP’s have populations with larger per capita incomes. Our hypothesis is that countries with more openness, measured by greater proportions of imports relative to GDP, will have higher per capita incomes. This is important, because if true, it implies that countries with relatively closed borders are limiting the income potential of their populations, which has broader implications for GDP. This analysis will address the effect of openness on per capita income.

To do so, we will utilize the Openness dataset which has 114 observations with 12 total variables each. There were no null values. The original dataset includes six untransformed variables and six transformed variables. Per capita income (pcinc) is our dependent variable. We will explore the remaining five variables and their transformations, if relevant. Then, we will explore several Ordinary Least Squares (OLS) models before selecting a “best” single-predictor model.

✓ Question 1

```

!pip install wooldridge
import wooldridge as woo
data = woo.data('openness')

```

```
Requirement already satisfied: wooldridge in /usr/local/lib/python3.10/dist-packages (0.4.5)
Requirement already satisfied: pandas in /usr/local/lib/python3.10/dist-packages (from wooldridge) (2.2.2)
Requirement already satisfied: numpy>=1.22.4 in /usr/local/lib/python3.10/dist-packages (from pandas->wooldridge) (1.26.4)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.10/dist-packages (from pandas->wooldridge) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/dist-packages (from pandas->wooldridge) (2024.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.10/dist-packages (from pandas->wooldridge) (2024.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.10/dist-packages (from python-dateutil>=2.8.2->pandas->wooldridge) (1.16.0)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 114 entries, 0 to 113
Data columns (total 12 columns):
 #   Column      Non-Null Count  Dtype  
---  -
 0   open        114 non-null    float64
 1   inf         114 non-null    float64
 2   pcinc       114 non-null    int64   
 3   land        114 non-null    float64
 4   oil         114 non-null    int64   
 5   good        114 non-null    int64   
 6   lpcinc      114 non-null    float64
 7   lland       114 non-null    float64
 8   lopen       114 non-null    float64
 9   linf        114 non-null    float64
10  opendec     114 non-null    float64
11  linfdec     114 non-null    float64
dtypes: float64(9), int64(3)
memory usage: 10.8 KB
```

```
#check index of data - matches what we would expect from the above ".info()"
data.index
```

```
RangeIndex(start=0, stop=114, step=1)
```

```
#check shape of data - matches what we would expect from the above ".info()"
data.shape
```

```
(114, 12)
```

```
#check head of dataset -
#there are 5 unique predictor variables
#there are 5 log transformation variables - one for each predictor variables
#there are is 1 depedent variable (%), and another column with the same % value as a decimal

data.head()
```



	open	inf	pcinc	land	oil	good	lpcinc	lland	lopen	linf	opendec	linfdec
0	31.400000	9.9	1998	919595.0	0	0	7.599902	13.731688	3.446808	2.292535	0.314	-2.312635
1	9.400000	117.0	4342	1072067.0	0	1	8.376090	13.885099	2.240710	4.762174	0.094	0.157004
2	16.700001	9.5	8349	2966150.0	0	1	9.029897	14.902776	2.815409	2.251292	0.167	-2.353878
3	35.599998	5.0	8230	32375.0	0	1	9.015541	10.385141	3.572345	1.609438	0.356	-2.995732
4	91.000000	4.8	9185	240.0	1	0	9.125327	5.480639	4.510859	1.568616	0.910	-3.036554

```
data.describe()
```

```
# The values of pcinc (per capita income) range between 224 and 25646 and the values of land range between 1.220000e+02 and 3.851809e+06. Based on
```



	open	inf	pcinc	land	oil	good	lpcinc	lland	lopen	linf	opendec	linfdec
count	114.000000	114.000000	114.000000	1.140000e+02	114.000000	114.000000	114.000000	114.000000	114.000000	114.000000	114.000000	114.000000
mean	37.078947	17.264035	3790.035088	2.988550e+05	0.061404	0.543860	7.658798	11.125916	3.441141	2.501944	0.370789	-2.103226
std	23.753526	23.997312	4155.718648	6.491452e+05	0.241129	0.500272	1.137958	2.086968	0.592017	0.709202	0.237535	0.709202
min	7.400000	3.600000	224.000000	1.220000e+02	0.000000	0.000000	5.411646	4.804021	2.001480	1.280934	0.074000	-3.324236
25%	21.824999	8.325000	764.000000	2.564900e+04	0.000000	0.000000	6.637573	10.152035	3.083054	2.119250	0.218250	-2.485921
50%	32.700001	10.650000	2005.000000	9.512850e+04	0.000000	1.000000	7.603393	11.462941	3.487356	2.365549	0.327000	-2.239621
75%	45.700000	14.800000	5595.000000	2.574648e+05	0.000000	1.000000	8.627128	12.458312	3.822091	2.694558	0.457000	-1.910612
max	163.800003	206.699997	25646.000000	3.851809e+06	1.000000	1.000000	10.152143	15.164054	5.098646	5.331268	1.638000	0.726098

```
#data descriptors including skew and kurtosis
```

```
for i in data.columns:
```

```
    print(i)
```

```
    print(data[i].describe())
```

```
    data_skew = stats.skew(data[i])
```

```
    data_kurt = stats.kurtosis(data[i])
```

```
    print(f'skew\t {data_skew:.5}')
```

```
    print(f'kurtosis {data_kurt:.6}')
```

```
    print()
```



```
open
count    114.000000
mean      37.078947
std       23.753526
min        7.400000
25%       21.824999
50%       32.700001
75%       45.700000
max       163.800003
```

```
Name: open, dtype: float64
skew      2.0879
kurtosis  6.88658
```

```
inf
count      114.000000
mean       17.264035
std        23.997312
min         3.600000
25%         8.325000
50%        10.650000
75%        14.800000
max        206.699997
Name: inf, dtype: float64
skew       5.3413
kurtosis  35.1035
```

```
pcinc
count      114.000000
mean       3790.035088
std        4155.718648
min         224.000000
25%         764.000000
50%        2005.000000
75%        5595.000000
max       25646.000000
Name: pcinc, dtype: float64
skew       2.117
kurtosis  6.55866
```

```
land
count      1.140000e+02
mean       2.988550e+05
std        6.491452e+05
min        1.220000e+02
25%        2.564900e+04
50%        9.512850e+04
75%        2.574648e+05
max        3.851809e+06
Name: land, dtype: float64
skew       4.088
kurtosis  17.2242
```

```
oil
count      114.000000
mean       0.061404
std        0.241129
min         0.000000
max         0.000000
```

The above analysis outlines the mean, standard deviation, 25th percentile, median, 75% percentile, minimum value, and maximum value for each of our variables. A few figures stand out.

First, oil has a mean of 0.06. Oil is a dummy variable that =1 when the country/observation is a major oil producer. A mean significantly less than 0.5 means that oil data is skewed towards non-major oil producers. Upon deeper examination of our data, we learn that only 7 countries in our data set are major oil producers.

Second, we notice that land and per capita income have much higher magnitudes than openness and inflation (both measured as percentages). Land area, total square miles of a country, has a mean of 300k sq. miles and a max value of 3m sq. miles. Per capita income has a median of 2005usd and a max of 25,000 usd.

We also notice that both inflation and openness, despite being percentages, have maximum values greater than 100%. While slightly abnormal, we do not believe these values are incorrect. Some countries experience hyperinflation, and given the data is '73-, it is reasonable to assume that a country could experience hyperinflation of 206% over a long period of time. Openness is total imports over total GDP. A max value of 164% would mean that a country imports goods equal to 64% of its GDP. This is a large import share, but not unreasonable.

```
#creating Quantile plots Untransformed
fig2, ax2 = plt.subplots(3, 2, figsize=(10,12))

sm.qqplot(data['open'], line='s', ax=ax2[0, 0])
ax2[0, 0].set_title('Q-Q Plot of Openness (imports as % of GDP)')

sm.qqplot(data['inf'], line='s', ax=ax2[0, 1])
ax2[0, 1].set_title('Q-Q Plot of Inflation (avg. annual inflation)')

sm.qqplot(data['pcinc'], line='s', ax=ax2[1, 0])
ax2[1, 0].set_title('Q-Q Plot of Per Capita Income')

sm.qqplot(data['land'], line='s', ax=ax2[1, 1])
ax2[1, 1].set_title('Q-Q Plot of Land Area (square miles)')

sm.qqplot(data['good'], line='s', ax=ax2[2, 0])
ax2[2, 0].set_title('Q-Q Plot of Good Data')

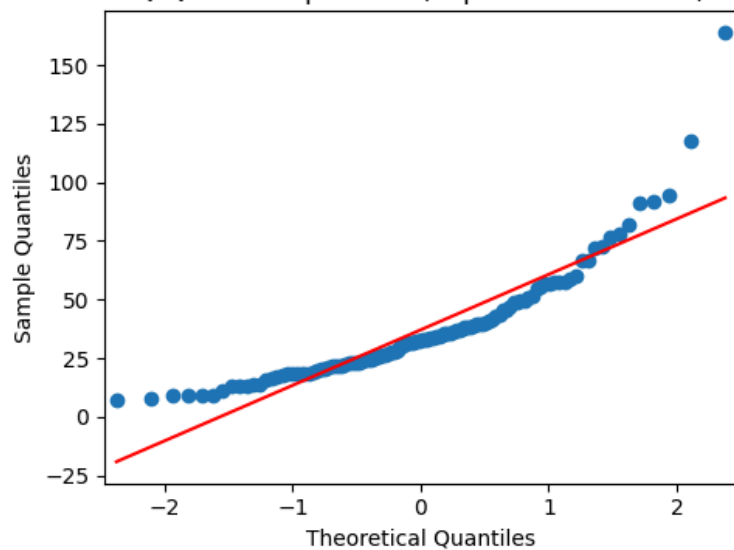
sm.qqplot(data['oil'], line='s', ax=ax2[2, 1])
ax2[2, 1].set_title('Q-Q Plot of Major Oil Producers')

plt.tight_layout()

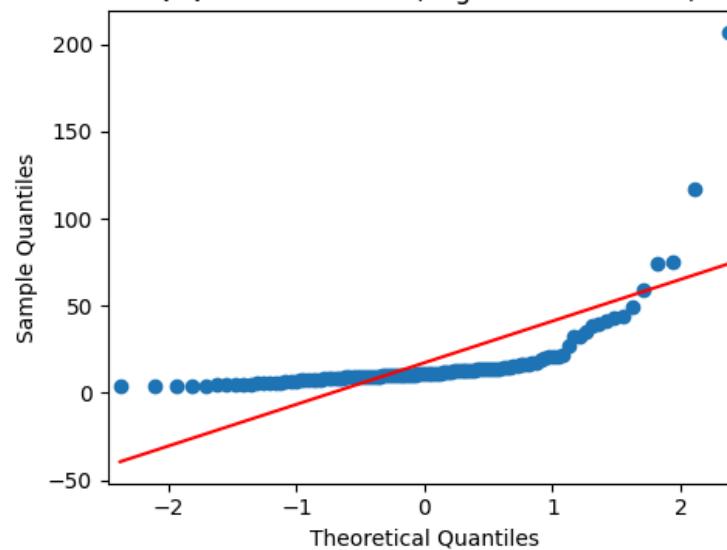
plt.show()
```



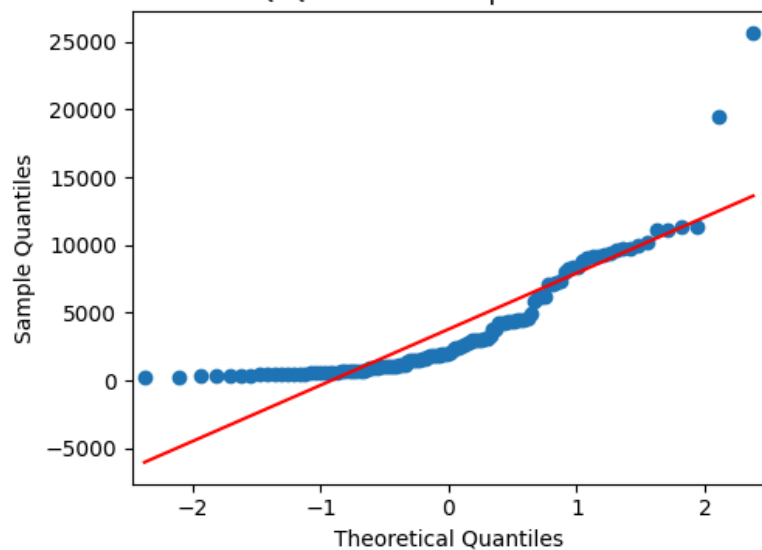

Q-Q Plot of Openness (imports as % of GDP)



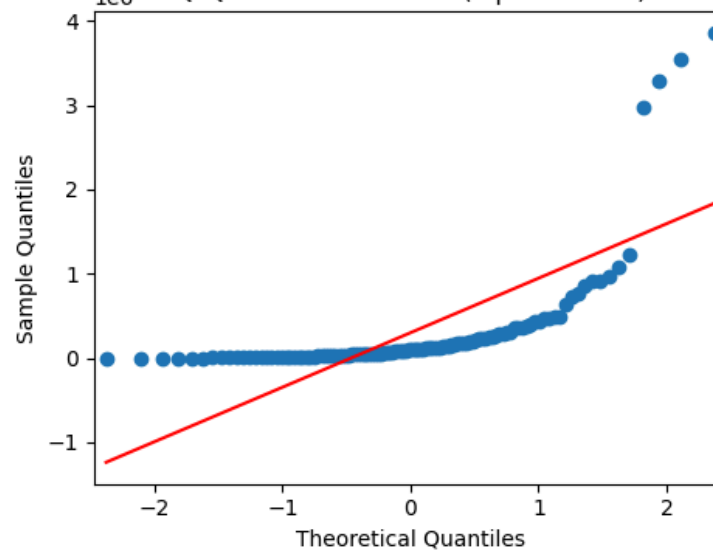
Q-Q Plot of Inflation (avg. annual inflation)



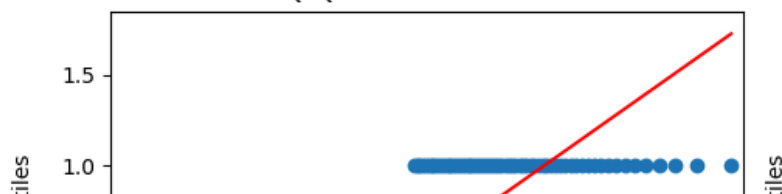
Q-Q Plot of Per Capita Income



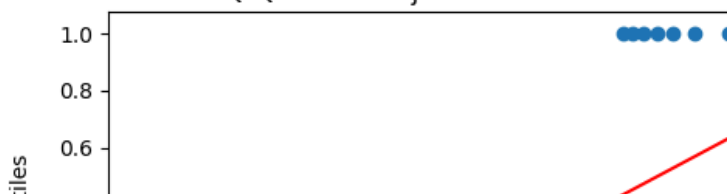
Q-Q Plot of Land Area (square miles)

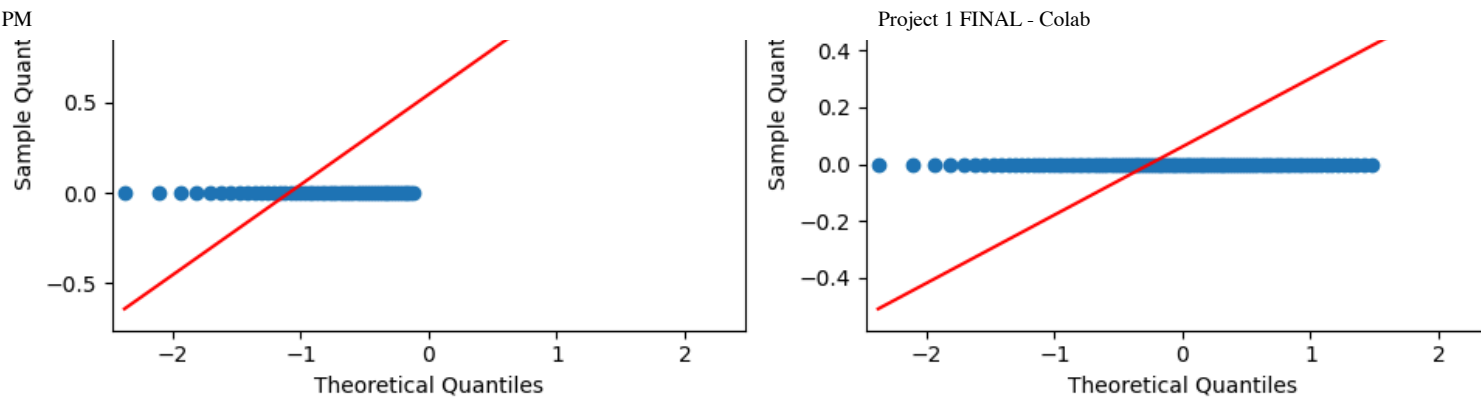


Q-Q Plot of Good Data



Q-Q Plot of Major Oil Producers





On testing the skewness and kurtosis using QQ Plots, we see that the inflation (inf), land area (total sq. miles of a country), per capita income (pcinc), and openness (imports as a % of GDP) are all skewed (observations at the ends of the blue are calling above and below the red line). If not for outliers on the upper right of the graphs, inflation, per capita income, and openness might also demonstrate slight kurtosis. This all suggests non-normality in our data which we plan to explore further below with QQ plots of transformed variables.

Oil (=1 if major oil producers) and good data (=1 if good, high quality data) are unique cases, given they are both dummy variables. Given there are only two possible outcomes, these QQ plots look as expected. Good data looks relatively equally distributed in terms of relatively equal 0's and 1's. Oil on the other hand looks skewed towards 0.

We confirm based on our data description table above that oil =1 for only 7 observations. This means we should be wary about any generalizations coming from oil later on in our analysis and regressions. Good data is less concerning, given the relatively even split. However, Wooldridge does not offer an explanation of what constitutes "good" data. So, it is concerning that about half of our observations are not "good" data.

```
#quantile plots Transformed
fig2, ax2 = plt.subplots(2, 2, figsize=(10,12))

sm.qqplot(data['lopen'], line='s', ax=ax2[0,0])
ax2[0,0].set_title('Q-Q Plot of Log-Openness (imports as % of GDP)')

sm.qqplot(data['linf'], line='s', ax=ax2[0,1])
ax2[0,1].set_title('Q-Q Plot of Log-Inflation (avg. annual inflation)')

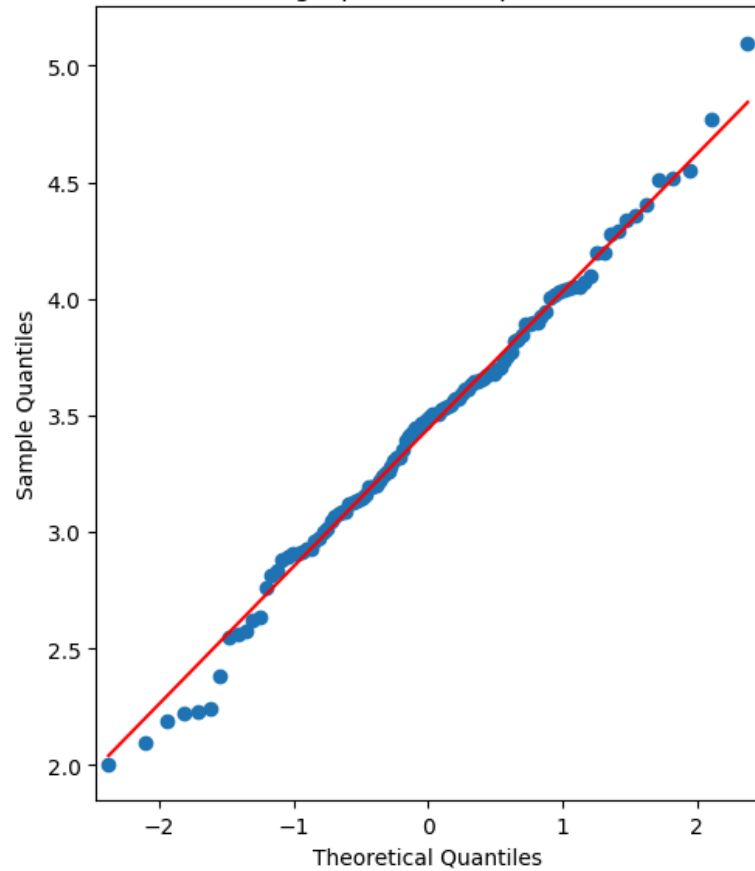
sm.qqplot(data['lpcinc'], line='s', ax=ax2[1,1])
ax2[1,1].set_title('Q-Q Plot of Log-Per Capita Income')

sm.qqplot(data['lland'], line='s', ax=ax2[1,0])
ax2[1,0].set_title('Q-Q Plot of Log-Land Area')

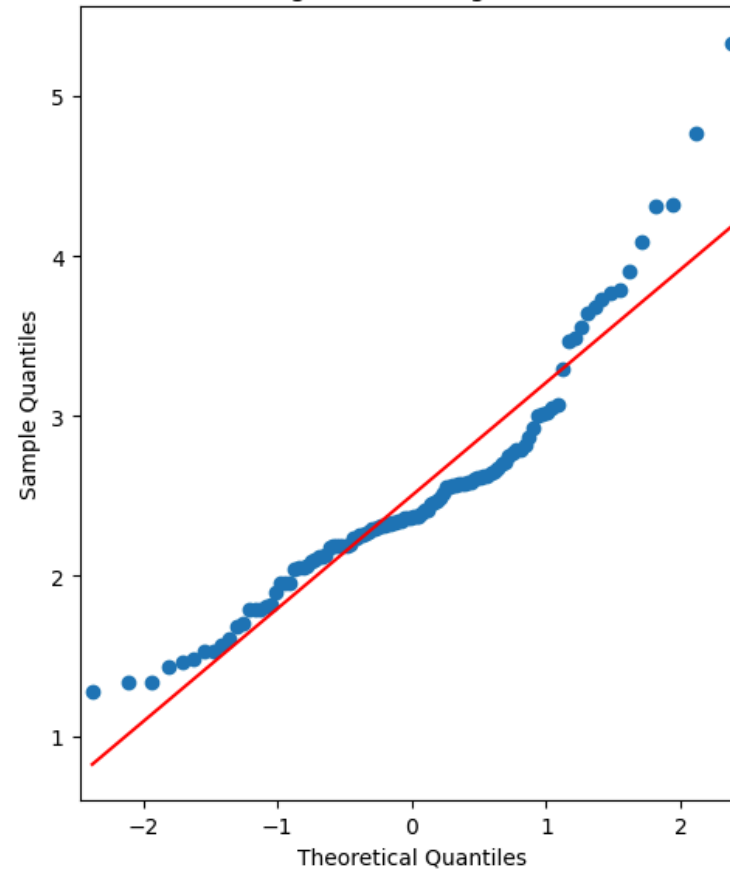
plt.tight_layout()
plt.show()
```



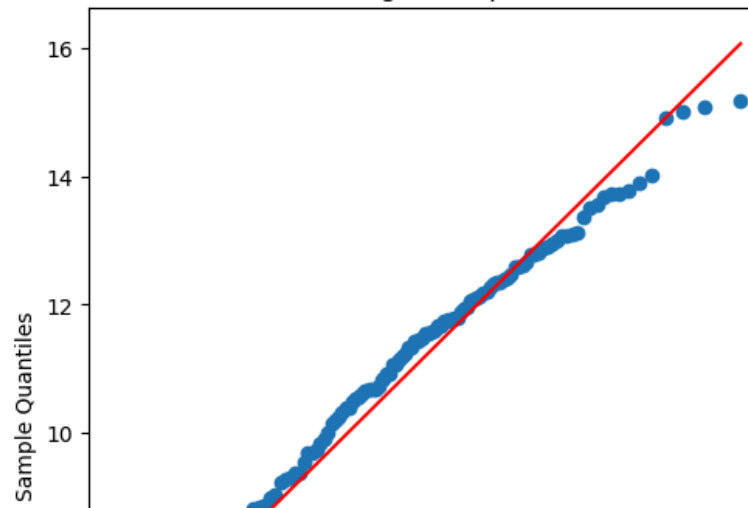
Q-Q Plot of Log-Openness (imports as % of GDP)



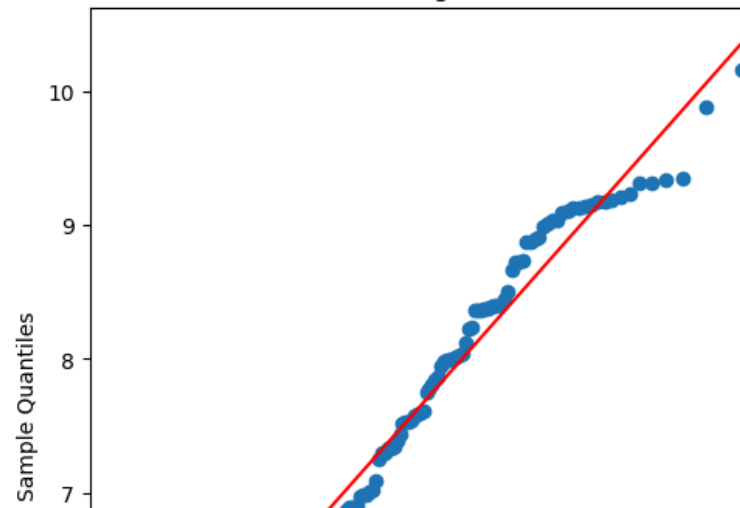
Q-Q Plot of Log-Inflation (avg. annual inflation)

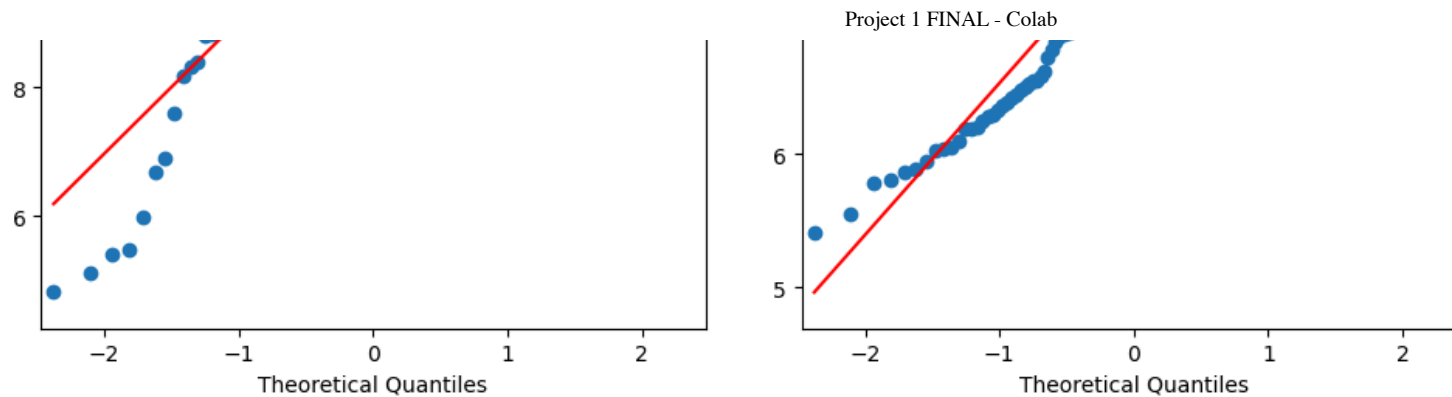


Q-Q Plot of Log-Per Capita Income



Q-Q Plot of Log-Land Area





Above, we created QQ plots of log-transformations on openness, inflation, per capita income, and land area. In the previous set of QQ plots, those on untransformed variables, no variables were close to normal.

We find that all four variables, when log transformed, are closer to normal distributions than their untransformed distributions. We will further validate that log is the best fit transformation using a fitter test down below (response to question 1D).

This transformation is also helpful to our future modeling due to magnitude. Openness and inflation are both measured as percentages. Notably, openness can be more than 100% if a country imports more than it produces. The max value for openness is 164%.

The two variables with much larger magnitudes are land area and per capita income. Land area, total square miles of a country, has a mean of 300k sq. miles and a max value of 3m sq. miles. Per capita income has a median of 2005usd and a max of 25k usd. Log transformations, in addition to normalizing the distribution of the data, also help normalize the magnitude in relation to our other percentage values.

```
#correlation matrix untransformed
```

```
plt.figure(figsize=(17,7))
C = data[['pcinc','open','inf','land', 'oil', 'good']].corr()
print(C)
sns.heatmap(C, annot=True, cmap="Blues")
```

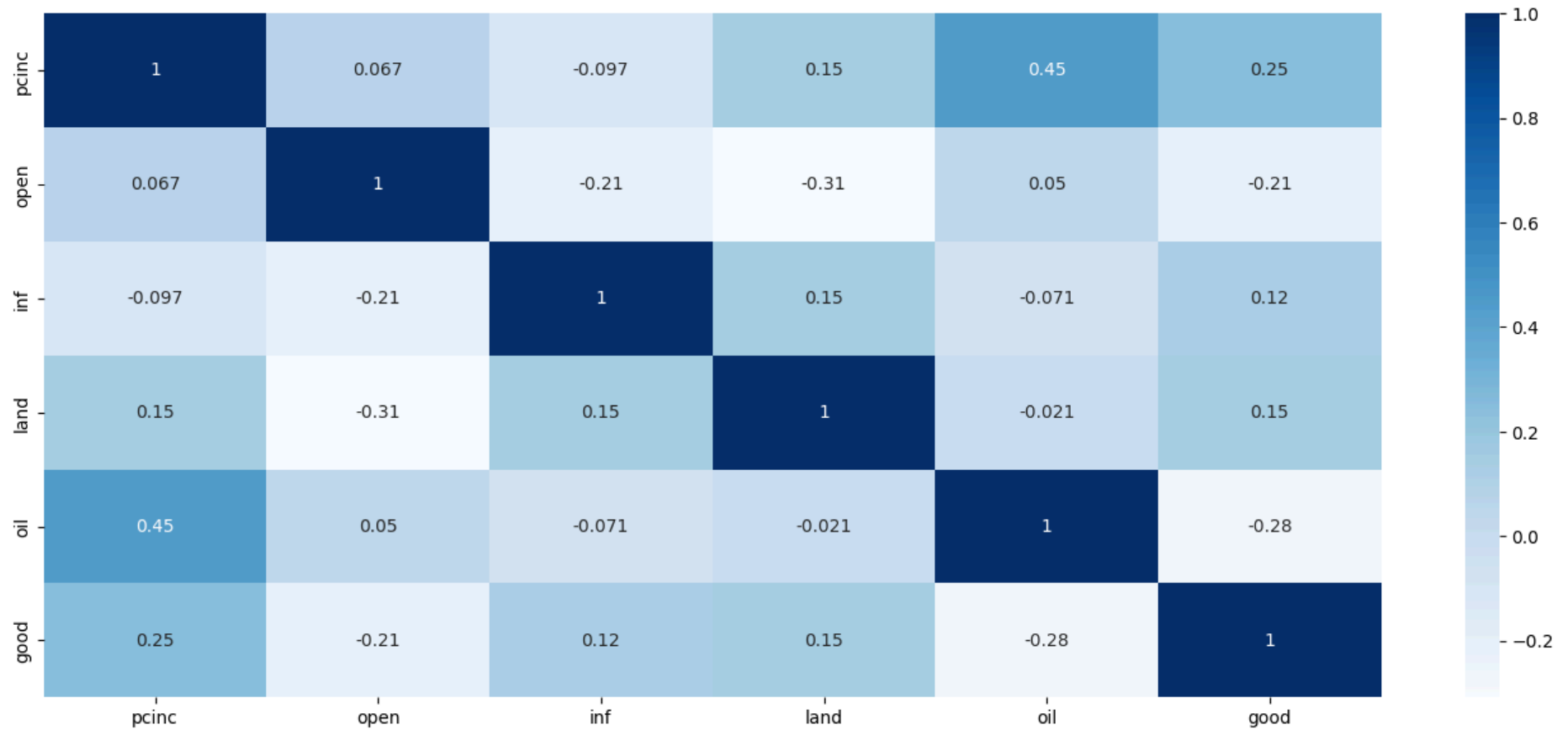


```

pcinc  1.000000  0.066869 -0.096602  0.150693  0.450237  0.250504
open   0.066869  1.000000 -0.212768 -0.308256  0.050288 -0.210153
inf    -0.096602 -0.212768  1.000000  0.151826 -0.070883  0.122093
land   0.150693 -0.308256  0.151826  1.000000 -0.020704  0.150936
oil    0.450237  0.050288 -0.070883 -0.020704  1.000000 -0.279287
good   0.250504 -0.210153  0.122093  0.150936 -0.279287  1.000000

```

<Axes: >



#correlation matrix

```

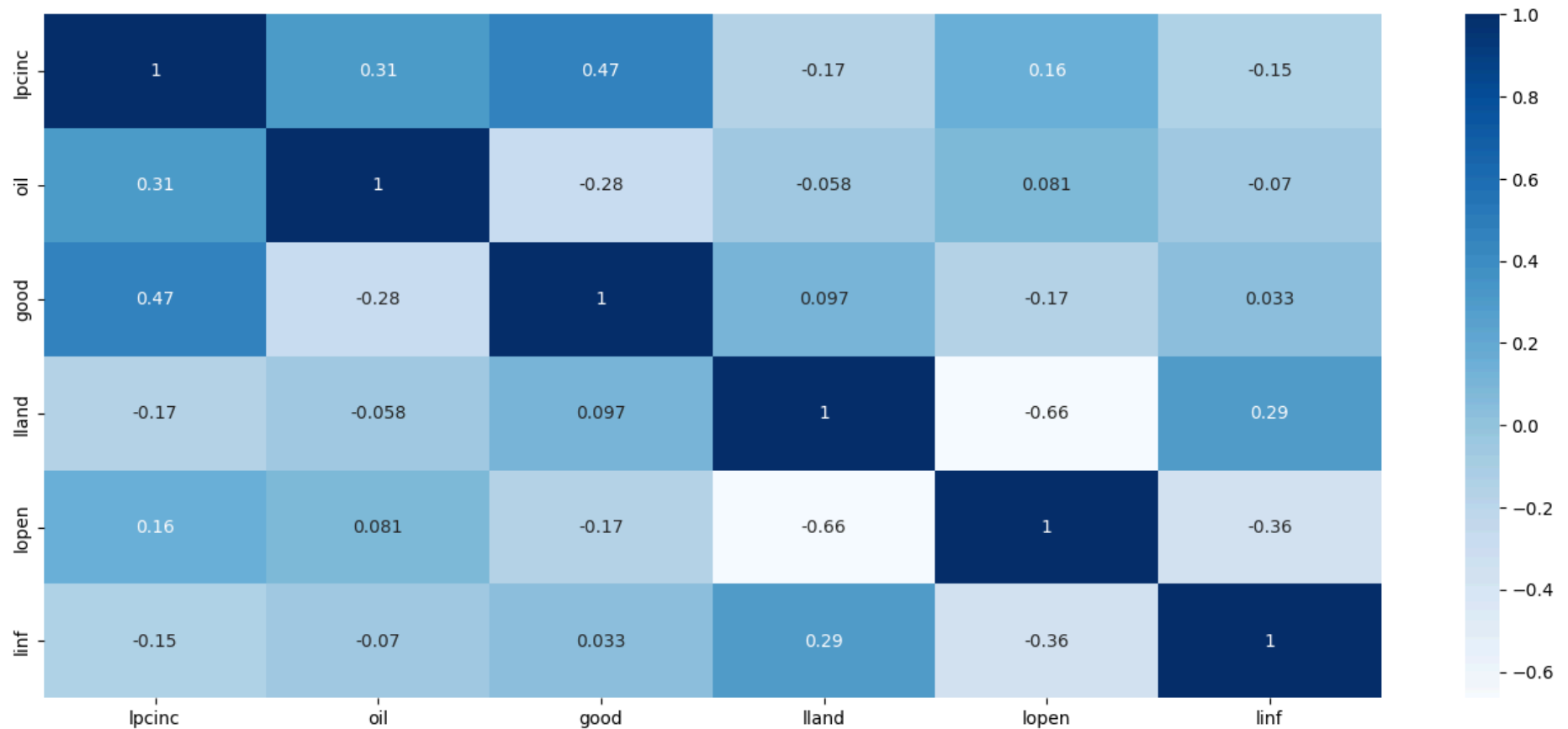
plt.figure(figsize=(17,7))
C = data[['pcinc', 'oil', 'good', 'land', 'open', 'inf']].corr()
print(C)
sns.heatmap(C, annot=True, cmap="Blues")

```

```

↕
lpcinc  lpcinc  oil      good    lland    lopen    linf
lpcinc  1.000000  0.308808  0.473353 -0.171500  0.155171 -0.148979
oil     0.308808  1.000000 -0.279287 -0.057968  0.081043 -0.069534
good    0.473353 -0.279287  1.000000  0.097098 -0.171006  0.033270
lland   -0.171500 -0.057968  0.097098  1.000000 -0.664064  0.294978
lopen    0.155171  0.081043 -0.171006 -0.664064  1.000000 -0.358259
linf    -0.148979 -0.069534  0.033270  0.294978 -0.358259  1.000000
<Axes: >

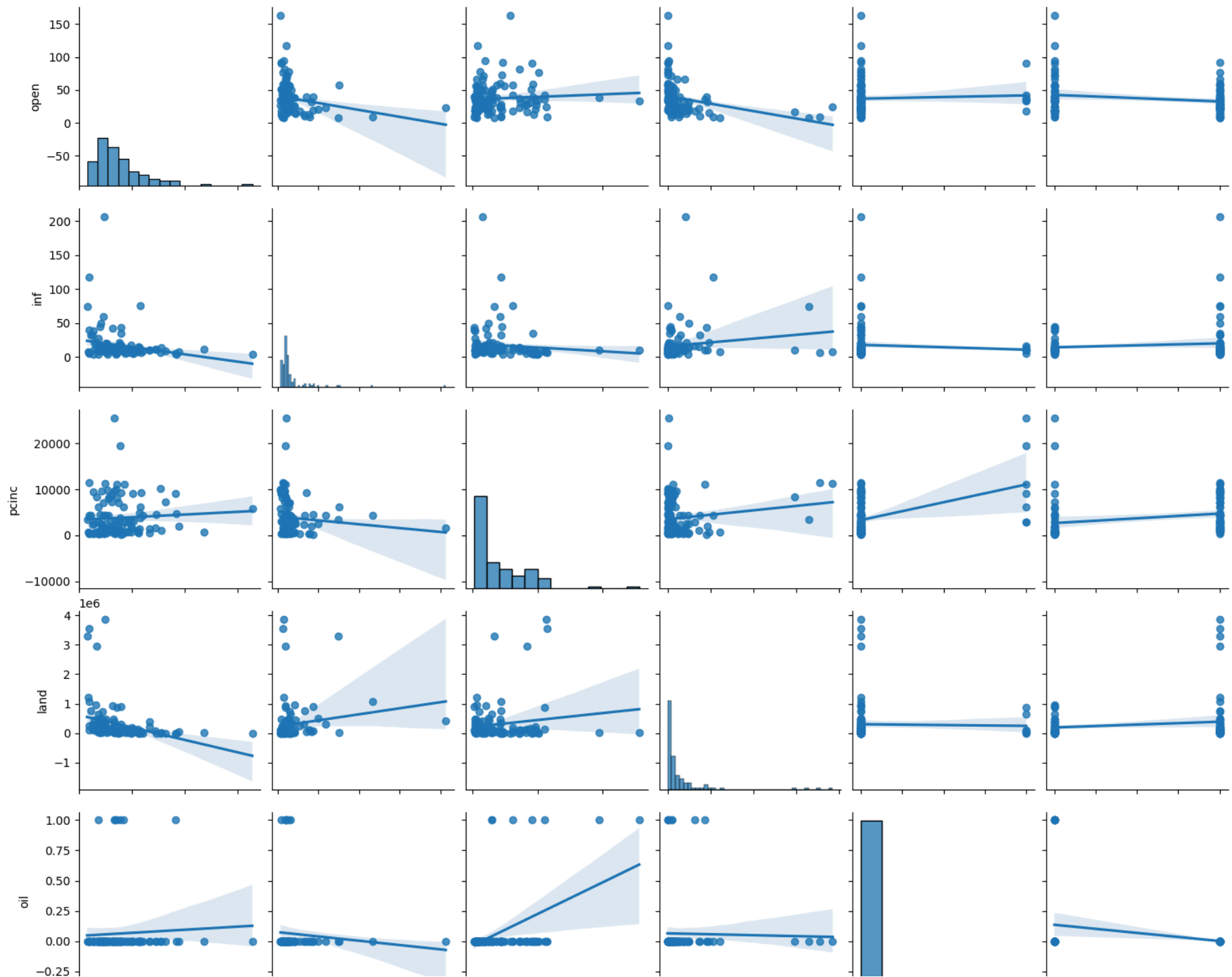
```

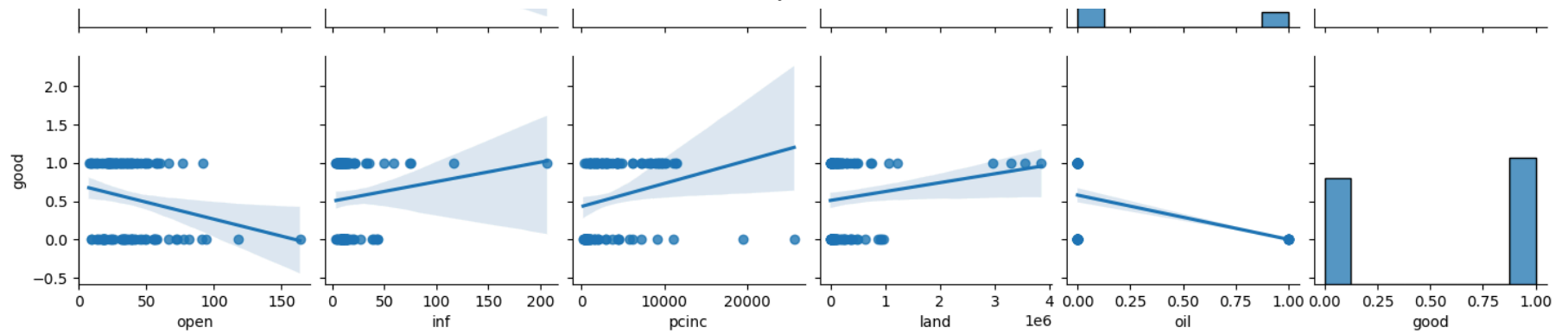


```
#create a pairplot of our unique variables
```

```
sns.pairplot(data[['open', 'inf', 'pcinc', 'land', 'oil', 'good']], kind='reg')
```

 <seaborn.axisgrid.PairGrid at 0x7e55907184f0>





```
#correletion matrix
```

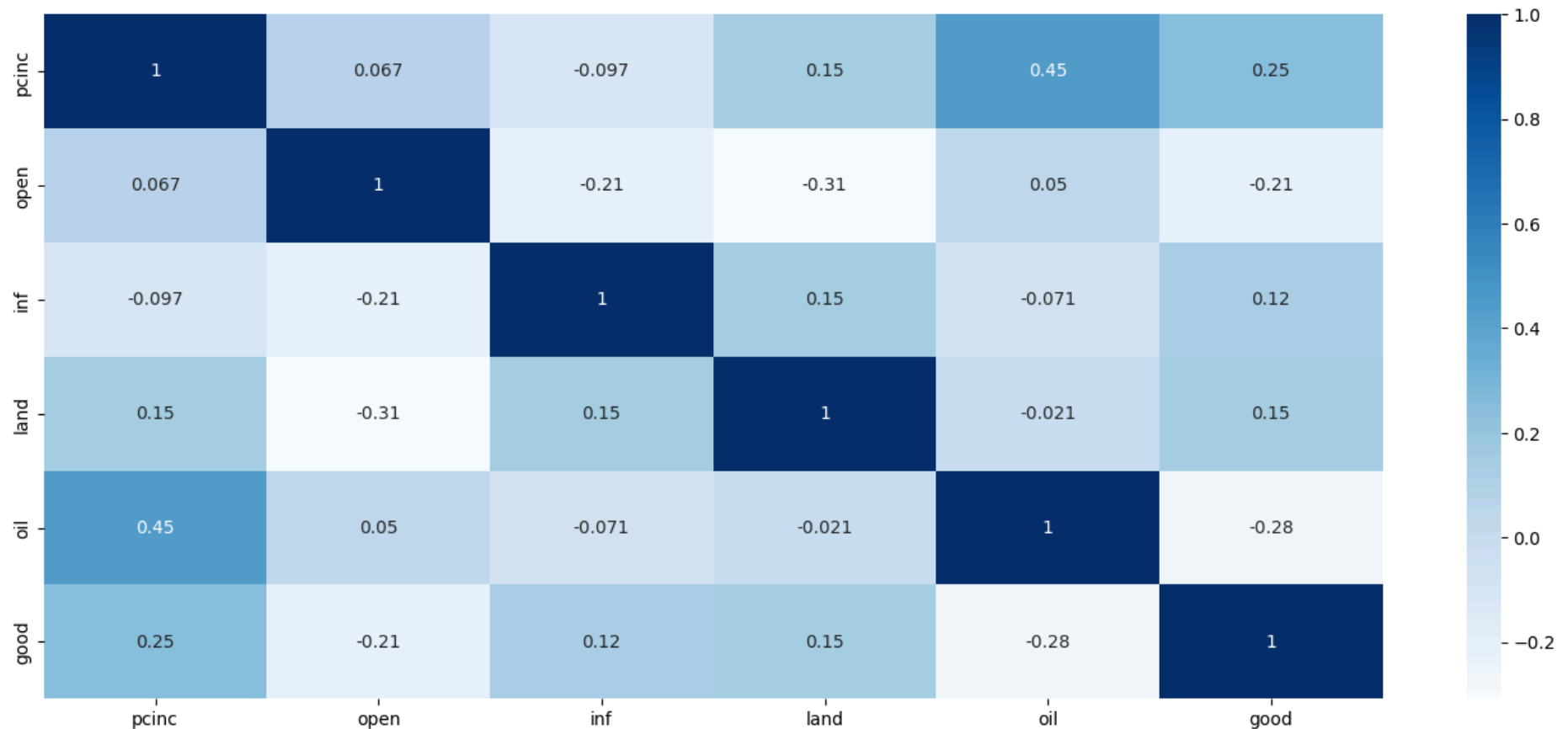
```
plt.figure(figsize=(17,7))
C = data[['pcinc','open','inf','land', 'oil', 'good']].corr()
print(C)
sns.heatmap(C, annot=True, cmap="Blues")
```



```

pcinc  1.000000  0.066869 -0.096602  0.150693  0.450237  0.250504
open   0.066869  1.000000 -0.212768 -0.308256  0.050288 -0.210153
inf    -0.096602 -0.212768  1.000000  0.151826 -0.070883  0.122093
land   0.150693 -0.308256  0.151826  1.000000 -0.020704  0.150936
oil    0.450237  0.050288 -0.070883 -0.020704  1.000000 -0.279287
good   0.250504 -0.210153  0.122093  0.150936 -0.279287  1.000000
<Axes: >

```



```
pip install pandas-profiling
```

```

Requirement already satisfied: pandas-profiling in /usr/local/lib/python3.10/dist-packages (3.6.6)
Requirement already satisfied: ydata-profiling in /usr/local/lib/python3.10/dist-packages (from pandas-profiling) (4.11.0)
Requirement already satisfied: scipy<1.14,>=1.4.1 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling->pandas-profiling) (1.13.1)
Requirement already satisfied: pandas!=1.4.0,<3,>1.1 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling->pandas-profiling) (2.2.1)
Requirement already satisfied: matplotlib<3.10,>=3.5 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling->pandas-profiling) (3.7.1)
Requirement already satisfied: pydantic>=2 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling->pandas-profiling) (2.9.2)
Requirement already satisfied: PyYAML<6.1,>=5.0.0 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling->pandas-profiling) (6.0.2)
Requirement already satisfied: Jinja2<3.2,>=2.11.1 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling->pandas-profiling) (3.1.4)

```

```

Requirement already satisfied: visions<0.7.7,>=0.7.5 in /usr/local/lib/python3.10/dist-packages (from visions[type_image_path]<0.7.7,>=0.7.5->)
Requirement already satisfied: numpy<2.2,>=1.16.0 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling->pandas-profiling) (1.26.4)
Requirement already satisfied: htmlmin==0.1.12 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling->pandas-profiling) (0.1.12)
Requirement already satisfied: phik<0.13,>=0.11.1 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling->pandas-profiling) (0.12.4)
Requirement already satisfied: requests<3,>=2.24.0 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling->pandas-profiling) (2.32.3)
Requirement already satisfied: tqdm<5,>=4.48.2 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling->pandas-profiling) (4.66.5)
Requirement already satisfied: seaborn<0.14,>=0.10.1 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling->pandas-profiling) (0.13)
Requirement already satisfied: multimethod<2,>=1.4 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling->pandas-profiling) (1.12)
Requirement already satisfied: statsmodels<1,>=0.13.2 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling->pandas-profiling) (0.1)
Requirement already satisfied: typeguard<5,>=3 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling->pandas-profiling) (4.3.0)
Requirement already satisfied: imagehash==4.3.1 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling->pandas-profiling) (4.3.1)
Requirement already satisfied: wordcloud==1.9.3 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling->pandas-profiling) (1.9.3)
Requirement already satisfied: dacite>=1.8 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling->pandas-profiling) (1.8.1)
Requirement already satisfied: numba<1,>=0.56.0 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling->pandas-profiling) (0.60.0)
Requirement already satisfied: PyWavelets in /usr/local/lib/python3.10/dist-packages (from imagehash==4.3.1->ydata-profiling->pandas-profiling)
Requirement already satisfied: pillow in /usr/local/lib/python3.10/dist-packages (from imagehash==4.3.1->ydata-profiling->pandas-profiling) (10)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.10/dist-packages (from jinja2<3.2,>=2.11.1->ydata-profiling->pandas-p)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib<3.10,>=3.5->ydata-profiling->panda)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.10/dist-packages (from matplotlib<3.10,>=3.5->ydata-profiling->pandas-p)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib<3.10,>=3.5->ydata-profiling->panda)
Requirement already satisfied: kiwisolver>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib<3.10,>=3.5->ydata-profiling->panda)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib<3.10,>=3.5->ydata-profiling->pandas)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib<3.10,>=3.5->ydata-profiling->panda)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.10/dist-packages (from matplotlib<3.10,>=3.5->ydata-profiling->p)
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in /usr/local/lib/python3.10/dist-packages (from numba<1,>=0.56.0->ydata-profiling->)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/dist-packages (from pandas!=1.4.0,<3,>1.1->ydata-profiling->pandas-p)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.10/dist-packages (from pandas!=1.4.0,<3,>1.1->ydata-profiling->pandas-p)
Requirement already satisfied: joblib>=0.14.1 in /usr/local/lib/python3.10/dist-packages (from phik<0.13,>=0.11.1->ydata-profiling->pandas-pro)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.10/dist-packages (from pydantic>=2->ydata-profiling->pandas-p)
Requirement already satisfied: pydantic-core==2.23.4 in /usr/local/lib/python3.10/dist-packages (from pydantic>=2->ydata-profiling->pandas-pro)
Requirement already satisfied: typing-extensions>=4.6.1 in /usr/local/lib/python3.10/dist-packages (from pydantic>=2->ydata-profiling->pandas-p)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests<3,>=2.24.0->ydata-profiling-)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests<3,>=2.24.0->ydata-profiling->pandas-prof)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages (from requests<3,>=2.24.0->ydata-profiling->panda)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages (from requests<3,>=2.24.0->ydata-profiling->panda)
Requirement already satisfied: patsy>=0.5.6 in /usr/local/lib/python3.10/dist-packages (from statsmodels<1,>=0.13.2->ydata-profiling->pandas-p)
Requirement already satisfied: attrs>=19.3.0 in /usr/local/lib/python3.10/dist-packages (from visions<0.7.7,>=0.7.5->visions[type_image_path]<)
Requirement already satisfied: networkx>=2.4 in /usr/local/lib/python3.10/dist-packages (from visions<0.7.7,>=0.7.5->visions[type_image_path]<)
Requirement already satisfied: six in /usr/local/lib/python3.10/dist-packages (from patsy>=0.5.6->statsmodels<1,>=0.13.2->ydata-profiling->pan)

```

`pip install ydata-profiling`



```

Requirement already satisfied: ydata-profiling in /usr/local/lib/python3.10/dist-packages (4.11.0)
Requirement already satisfied: scipy<1.14,>=1.4.1 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling) (1.13.1)
Requirement already satisfied: pandas!=1.4.0,<3,>1.1 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling) (2.2.2)
Requirement already satisfied: matplotlib<3.10,>=3.5 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling) (3.7.1)
Requirement already satisfied: pydantic>=2 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling) (2.9.2)
Requirement already satisfied: PyYAML<6.1,>=5.0.0 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling) (6.0.2)
Requirement already satisfied: jinja2<3.2,>=2.11.1 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling) (3.1.4)
Requirement already satisfied: visions<0.7.7,>=0.7.5 in /usr/local/lib/python3.10/dist-packages (from visions[type_image_path]<0.7.7,>=0.7.5->)
Requirement already satisfied: numpy<2.2,>=1.16.0 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling) (1.26.4)
Requirement already satisfied: htmlmin==0.1.12 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling) (0.1.12)
Requirement already satisfied: phik<0.13,>=0.11.1 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling) (0.12.4)

```

```

Requirement already satisfied: requests<3,>=2.24.0 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling) (2.32.3)
Requirement already satisfied: tqdm<5,>=4.48.2 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling) (4.66.5)
Requirement already satisfied: seaborn<0.14,>=0.10.1 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling) (0.13.2)
Requirement already satisfied: multimethod<2,>=1.4 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling) (1.12)
Requirement already satisfied: statsmodels<1,>=0.13.2 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling) (0.14.4)
Requirement already satisfied: typeguard<5,>=3 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling) (4.3.0)
Requirement already satisfied: imagehash==4.3.1 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling) (4.3.1)
Requirement already satisfied: wordcloud>=1.9.3 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling) (1.9.3)
Requirement already satisfied: dacite>=1.8 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling) (1.8.1)
Requirement already satisfied: numba<1,>=0.56.0 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling) (0.60.0)
Requirement already satisfied: PyWavelets in /usr/local/lib/python3.10/dist-packages (from imagehash==4.3.1->ydata-profiling) (1.7.0)
Requirement already satisfied: pillow in /usr/local/lib/python3.10/dist-packages (from imagehash==4.3.1->ydata-profiling) (10.4.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.10/dist-packages (from jinja2<3.2,>=2.11.1->ydata-profiling) (3.0.2)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib<3.10,>=3.5->ydata-profiling) (1.3.0)
Requirement already satisfied: cycycler>=0.10 in /usr/local/lib/python3.10/dist-packages (from matplotlib<3.10,>=3.5->ydata-profiling) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib<3.10,>=3.5->ydata-profiling) (4.53.0)
Requirement already satisfied: kiwisolver>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib<3.10,>=3.5->ydata-profiling) (1.4.5)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib<3.10,>=3.5->ydata-profiling) (24.1)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib<3.10,>=3.5->ydata-profiling) (3.2.0)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.10/dist-packages (from matplotlib<3.10,>=3.5->ydata-profiling) (2.9.0)
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in /usr/local/lib/python3.10/dist-packages (from numba<1,>=0.56.0->ydata-profiling) (0.43.0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/dist-packages (from pandas!=1.4.0,<3,>1.1->ydata-profiling) (2024.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.10/dist-packages (from pandas!=1.4.0,<3,>1.1->ydata-profiling) (2024.2)
Requirement already satisfied: joblib>=0.14.1 in /usr/local/lib/python3.10/dist-packages (from phik<0.13,>=0.11.1->ydata-profiling) (1.4.2)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.10/dist-packages (from pydantic>=2->ydata-profiling) (0.7.0)
Requirement already satisfied: pydantic-core==2.23.4 in /usr/local/lib/python3.10/dist-packages (from pydantic>=2->ydata-profiling) (2.23.4)
Requirement already satisfied: typing-extensions>=4.6.1 in /usr/local/lib/python3.10/dist-packages (from pydantic>=2->ydata-profiling) (4.12.2)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests<3,>=2.24.0->ydata-profiling) (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests<3,>=2.24.0->ydata-profiling) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages (from requests<3,>=2.24.0->ydata-profiling) (2.2.3)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages (from requests<3,>=2.24.0->ydata-profiling) (2024.7.4)
Requirement already satisfied: patsy>=0.5.6 in /usr/local/lib/python3.10/dist-packages (from statsmodels<1,>=0.13.2->ydata-profiling) (0.5.6)
Requirement already satisfied: attrs>=19.3.0 in /usr/local/lib/python3.10/dist-packages (from visions<0.7.7,>=0.7.5->visions[type_image_path]<1.0.0) (25.1.0)
Requirement already satisfied: networkx>=2.4 in /usr/local/lib/python3.10/dist-packages (from visions<0.7.7,>=0.7.5->visions[type_image_path]<1.0.0) (3.4.2)
Requirement already satisfied: six in /usr/local/lib/python3.10/dist-packages (from patsy>=0.5.6->statsmodels<1,>=0.13.2->ydata-profiling) (1.16.0)

```

```

import pandas as pd
from ydata_profiling import ProfileReport

```

```
ProfileReport(data)
```



Summarize dataset: 100%

121/121 [00:59<00:00, 2.03it/s, Completed]

Generate report structure: 100%

1/1 [00:09<00:00, 9.18s/it]

Render HTML: 100%

1/1 [00:02<00:00, 2.00s/it]

Pandas Profiling Report

[Overview](#) [Variables](#) [Interactions](#) [Correlations](#) [Missing values](#) [Sample](#)

Overview

Brought to you by [YData](#)[Overview](#)[Alerts](#) 17[Reproduction](#)

Dataset statistics

Number of variables	12
Number of observations	114
Missing cells	0
Missing cells (%)	0.0%
Duplicate rows	0
Duplicate rows (%)	0.0%
Total size in memory	10.8 KiB
Average record size in memory	97.1 B

Variable types

Numeric	10
Categorical	2

Variables

[Select Columns](#)

Our pandas profiling validates the general observations we have made so far:

- 12 total variables
- 10 numerical variables
- 2 categorical, indicator variables
- No missing cells/null values
- Non-normal distributions of un-transformed variables
- Log transformations make our numeric variable distributions closer to normal

```
import rpy2.robjects as robjects
from rpy2.robjects import pandas2ri
```

```
pd.DataFrame.iteritems = pd.DataFrame.items
```

```
# Set things up to share objects between R and python environments
pandas2ri.activate()
```

```
# Have to convert panda dataframe to R dataframe, in this case df is our pandas dataframe
r_df = pandas2ri.py2rpy(data)
```

```
# Then stash our new R datastructure in a global environment we can grab later
robjects.globalenv['r_df'] = r_df
robjects.globalenv['df'] = pandas2ri.py2rpy(data)
```

```
%%R
install.packages("fitdistrplus")
```

```
⚠ WARNING:rpy2.rinterface_lib.callbacks:R[write to console]: Installing package into '/usr/local/lib/R/site-library'
(as 'lib' is unspecified)
```

```
WARNING:rpy2.rinterface_lib.callbacks:R[write to console]: trying URL 'https://cran.rstudio.com/src/contrib/fitdistrplus\_1.2-1.tar.gz'
```

```
WARNING:rpy2.rinterface_lib.callbacks:R[write to console]: Content type 'application/x-gzip'
```

```
WARNING:rpy2.rinterface_lib.callbacks:R[write to console]: length 1616872 bytes (1.5 MB)
```

```
WARNING:rpy2.rinterface_lib.callbacks:R[write to console]: =
WARNING:rpy2.rinterface_lib.callbacks:R[write to console]: =
WARNING:rpy2.rinterface_lib.callbacks:R[write to console]: =
WARNING:rpy2.rinterface_lib.callbacks:R[write to console]: =
WARNING:rpy2.rinterface_lib.callbacks:R[write to console]: =
WARNING:rpy2.rinterface_lib.callbacks:R[write to console]: =
WARNING:rpy2.rinterface_lib.callbacks:R[write to console]: =
WARNING:rpy2.rinterface_lib.callbacks:R[write to console]: =
WARNING:rpy2.rinterface_lib.callbacks:R[write to console]: =
WARNING:rpy2.rinterface_lib.callbacks:R[write to console]: =
WARNING:rpy2.rinterface_lib.callbacks:R[write to console]: =
WARNING:rpy2.rinterface_lib.callbacks:R[write to console]: =
```



```
title(sub = "Cullen-Frey for open")
```

```
descdist(df$inf, boot = 1000)
title(sub = "Cullen-Frey for inf")
```

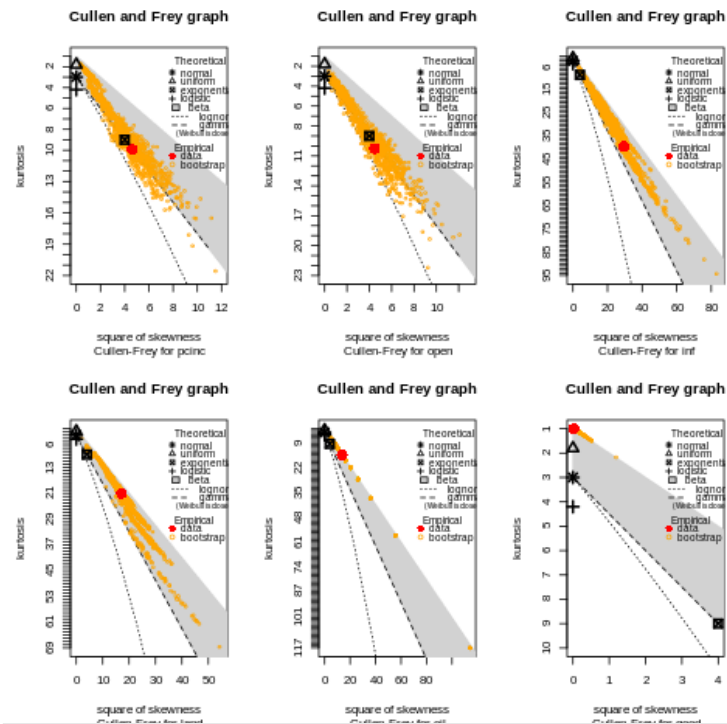
```
descdist(df$land, boot = 1000)
title(sub = "Cullen-Frey for land")
```

```
descdist(df$oil, boot = 1000)
title(sub = "Cullen-Frey for oil")
```

```
descdist(df$good, boot = 1000)
title(sub = "Cullen-Frey for good")
```

⚠ WARNING: rpy2.rinterface.lib.callbacks:R[write to console]: Loading required package: MASS

WARNING: rpy2.rinterface.lib.callbacks:R[write to console]: Loading required package: survival



We use the Cullen Frey graphs, generated by R in python, to see the possible distributions that our variables could fit. We see that the box with the X, exponential distribution, is the closest to red, real data dots for all our non-categorical variables. This validates the log transformations that we have identified as better fits already using tools above.

```
!pip install fitter
```



Collecting fitter

Downloading fitter-1.7.1-py3-none-any.whl.metadata (8.2 kB)

Requirement already satisfied: click<9.0.0,>=8.1.6 in /usr/local/lib/python3.10/dist-packages (from fitter) (8.1.7)

Requirement already satisfied: joblib<2.0.0,>=1.3.1 in /usr/local/lib/python3.10/dist-packages (from fitter) (1.4.2)

Collecting loguru<0.8.0,>=0.7.2 (from fitter)

Downloading loguru-0.7.2-py3-none-any.whl.metadata (23 kB)

Collecting matplotlib<4.0.0,>=3.7.2 (from fitter)

Downloading matplotlib-3.9.2-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (11 kB)

Requirement already satisfied: numpy<2.0.0,>=1.20.0 in /usr/local/lib/python3.10/dist-packages (from fitter) (1.26.4)

Requirement already satisfied: pandas<3.0.0,>=0.23.4 in /usr/local/lib/python3.10/dist-packages (from fitter) (2.2.2)

Collecting rich-click<2.0.0,>=1.7.2 (from fitter)

Downloading rich_click-1.8.3-py3-none-any.whl.metadata (7.9 kB)

Requirement already satisfied: scipy<2.0.0,>=0.18.0 in /usr/local/lib/python3.10/dist-packages (from fitter) (1.13.1)

Requirement already satisfied: tqdm<5.0.0,>=4.65.1 in /usr/local/lib/python3.10/dist-packages (from fitter) (4.66.5)

Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib<4.0.0,>=3.7.2->fitter) (1.3.0)

Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.10/dist-packages (from matplotlib<4.0.0,>=3.7.2->fitter) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib<4.0.0,>=3.7.2->fitter) (4.54.1)

Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib<4.0.0,>=3.7.2->fitter) (1.4.7)

Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib<4.0.0,>=3.7.2->fitter) (24.1)

Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.10/dist-packages (from matplotlib<4.0.0,>=3.7.2->fitter) (10.4.0)

Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib<4.0.0,>=3.7.2->fitter) (3.2.0)

Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.10/dist-packages (from matplotlib<4.0.0,>=3.7.2->fitter) (2.8.2)

Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/dist-packages (from pandas<3.0.0,>=0.23.4->fitter) (2024.2)

Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.10/dist-packages (from pandas<3.0.0,>=0.23.4->fitter) (2024.2)

Requirement already satisfied: rich>=10.7 in /usr/local/lib/python3.10/dist-packages (from rich-click<2.0.0,>=1.7.2->fitter) (13.9.3)

Requirement already satisfied: typing-extensions in /usr/local/lib/python3.10/dist-packages (from rich-click<2.0.0,>=1.7.2->fitter) (4.12.2)

Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.10/dist-packages (from python-dateutil>=2.7->matplotlib<4.0.0,>=3.7.2->fitter) (1.16.0)

Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.10/dist-packages (from rich>=10.7->rich-click<2.0.0,>=1.7.2->fitter) (3.0.0)

Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.10/dist-packages (from rich>=10.7->rich-click<2.0.0,>=1.7.2->fitter) (2.18.0)

Requirement already satisfied: mdurl~0.1 in /usr/local/lib/python3.10/dist-packages (from markdown-it-py>=2.2.0->rich>=10.7->rich-click<2.0.0,>=1.7.2->fitter) (0.1.2)

Downloading fitter-1.7.1-py3-none-any.whl (26 kB)

Downloading loguru-0.7.2-py3-none-any.whl (62 kB)

62.5/62.5 kB 3.4 MB/s eta 0:00:00

Downloading matplotlib-3.9.2-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (8.3 MB)

8.3/8.3 MB 54.8 MB/s eta 0:00:00

Downloading rich_click-1.8.3-py3-none-any.whl (35 kB)

Installing collected packages: loguru, matplotlib, rich-click, fitter

Attempting uninstall: matplotlib

Found existing installation: matplotlib 3.7.1

Uninstalling matplotlib-3.7.1:

Successfully uninstalled matplotlib-3.7.1

Successfully installed fitter-1.7.1 loguru-0.7.2 matplotlib-3.9.2 rich-click-1.8.3

```
from fitter import Fitter
```

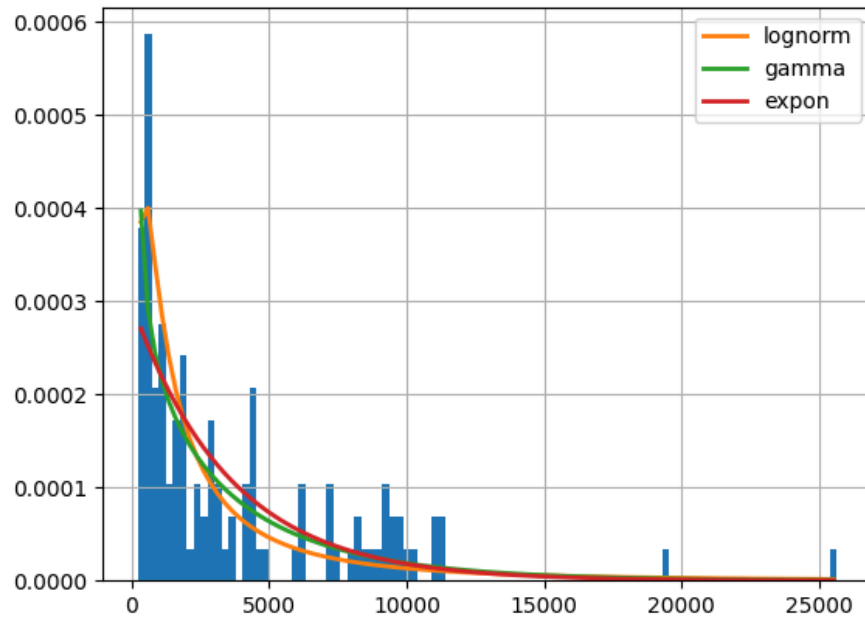
```
plt.figure()
data_fit = Fitter(data['pcinc'], distributions=['gamma', 'lognorm', 'expon'])
data_fit.fit()
plt.title("Fitting for plt")
```



```
data_fit.summary()
```



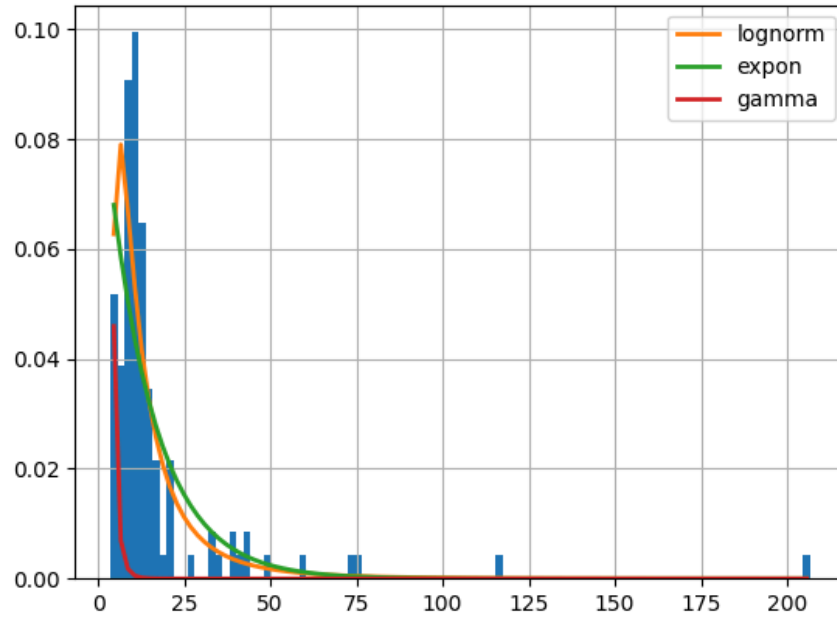
	sumsquare_error	aic	bic	kl_div	ks_statistic	ks_pvalue
lognorm	1.785474e-07	2289.707301	2297.915896	inf	0.087415	0.329105
gamma	2.070539e-07	2305.104095	2313.312690	inf	0.080900	0.422395
expon	2.565619e-07	2352.734520	2358.206917	inf	0.137688	0.023959



```
plt.figure()
data_fit = Fitter(data['inf'], distributions=['gamma', 'lognorm', 'expon'])
data_fit.fit()
plt.title("Fitting for inflation")
data_fit.summary()
```



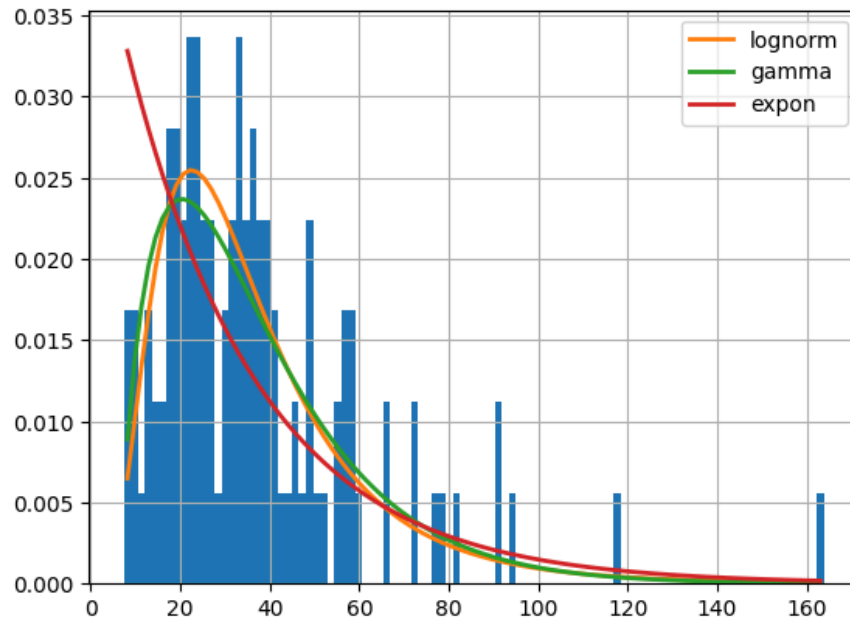
	sumsquare_error	aic	bic	kl_div	ks_statistic	ks_pvalue
lognorm	0.005921	1672.077722	1680.286317	inf	0.112459	1.034991e-01
expon	0.007621	2013.337181	2018.809578	inf	0.197196	2.325360e-04
gamma	0.025417	10030.007829	10038.216424	inf	0.898132	1.713438e-113



```
plt.figure()
data_fit = Fitter(data['open'], distributions=['gamma', 'lognorm', 'expon'])
data_fit.fit()
plt.title("Fitting for Open")
data_fit.summary()
```



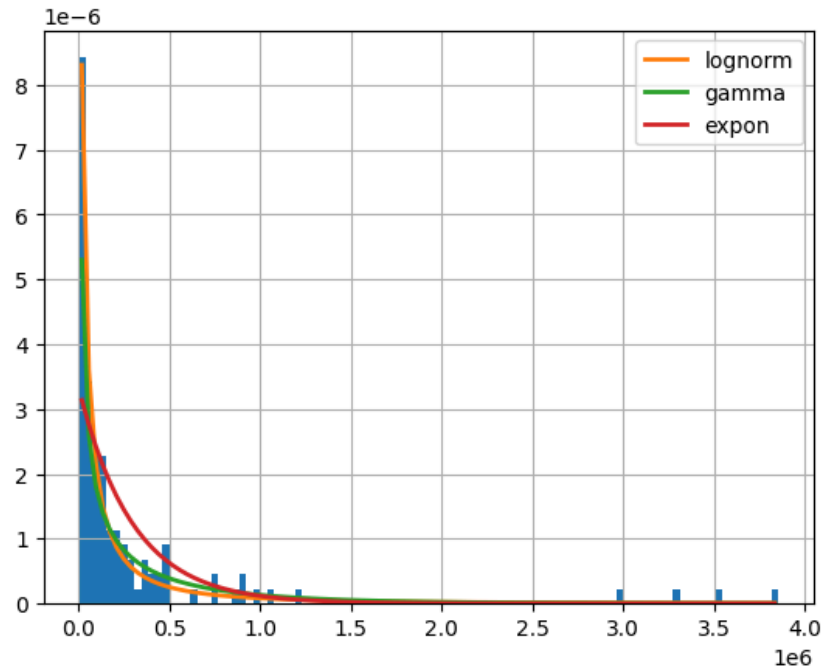
	sumsquare_error	aic	bic	kl_div	ks_statistic	ks_pvalue
lognorm	0.002385	1275.810938	1284.019533	inf	0.043815	0.974177
gamma	0.002516	1291.377695	1299.586290	inf	0.055864	0.849136
expon	0.004250	1209.060469	1214.532866	inf	0.164027	0.003786



```
plt.figure()
data_fit = Fitter(data['land'], distributions=['gamma', 'lognorm', 'expon'])
data_fit.fit()
plt.title("Fitting for land")
data_fit.summary()
```



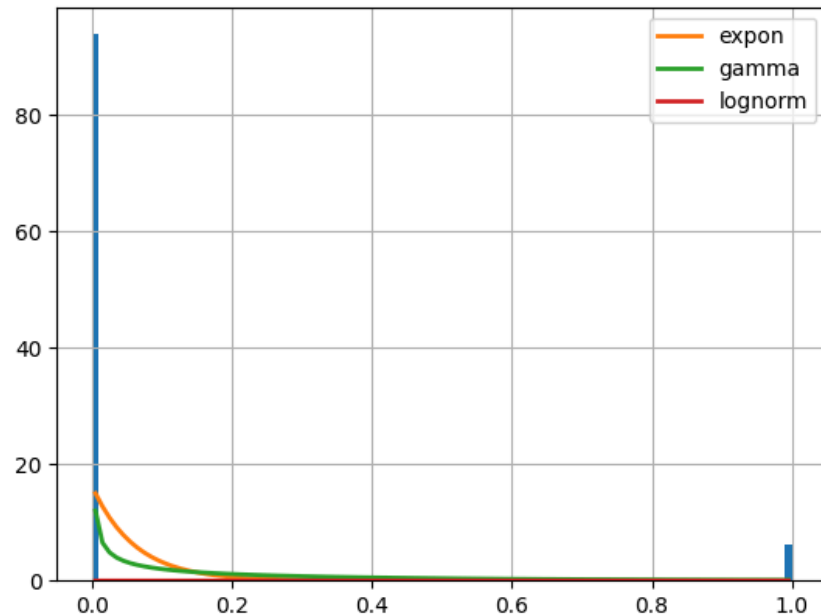
	sumsquare_error	aic	bic	kl_div	ks_statistic	ks_pvalue
lognorm	2.205192e-12	3423.701607	3431.910202	inf	0.071249	5.840519e-01
gamma	1.298622e-11	3463.616719	3471.825314	inf	0.081478	4.135272e-01
expon	3.268777e-11	3814.801951	3820.274348	inf	0.257034	3.953043e-07



```
plt.figure()
data_fit = Fitter(data['oil'], distributions=['gamma', 'lognorm', 'expon'])
data_fit.fit()
plt.title("Fitting for Oil")
data_fit.summary()
```



	sumsquare_error	aic	bic	kl_div	ks_statistic	ks_pvalue
expon	6839.965116	1074.513769	1079.986166	inf	0.938596	1.429138e-138
gamma	6897.294872	159.529019	167.737615	inf	0.938596	1.429138e-138
lognorm	8847.292529	2551.858096	2560.066692	inf	0.539534	1.387297e-31



To understand which distribution fits better, we use the fitter module in python for a fitter graph, where the summary returns the best-fitting probability distribution for a given dataset. All our variables best fit the lognormal distribution, except for our categorical dummies, which we will handle differently.

✓ Question 2

```
!pip install Boruta
```



```
Requirement already satisfied: Boruta in /usr/local/lib/python3.10/dist-packages (0.4.3)
Requirement already satisfied: numpy>=1.10.4 in /usr/local/lib/python3.10/dist-packages (from Boruta) (1.26.4)
Requirement already satisfied: scikit-learn>=0.17.1 in /usr/local/lib/python3.10/dist-packages (from Boruta) (1.5.2)
Requirement already satisfied: scipy>=0.17.0 in /usr/local/lib/python3.10/dist-packages (from Boruta) (1.13.1)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn>=0.17.1->Boruta) (1.4.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn>=0.17.1->Boruta) (3.5.0)
```

```

from sklearn.ensemble import RandomForestClassifier
from boruta import BorutaPy
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.ensemble import RandomForestRegressor

data_Y = data['pcinc'].values # Target variable: pcinc
data_X = data[['open', 'inf', 'land', 'oil', 'good']].values.astype(np.float32) # Potential predictors

np.int = np.int32
np.float = np.float64
np.bool = np.bool_

# If you want to speed up computation time, consider parallel computing (n_jobs=-1)
rfc = RandomForestRegressor(max_depth=5)
boruta = BorutaPy(rfc, n_estimators='auto', verbose=2, random_state=10, two_step=True) # `two_step` uses Bonferroni

# Run algorithm, X and y must be numpy arrays
boruta.fit(data_X, data_Y)

# Print confirmed columns
print(data[['open', 'inf', 'land', 'oil', 'good']].columns[boruta.support_])

```

```

➦ Iteration:      1 / 100
  Confirmed:      0
  Tentative:      5
  Rejected:       0
  Iteration:      2 / 100
  Confirmed:      0
  Tentative:      5
  Rejected:       0
  Iteration:      3 / 100
  Confirmed:      0
  Tentative:      5
  Rejected:       0
  Iteration:      4 / 100
  Confirmed:      0
  Tentative:      5
  Rejected:       0
  Iteration:      5 / 100
  Confirmed:      0
  Tentative:      5
  Rejected:       0
  Iteration:      6 / 100
  Confirmed:      0
  Tentative:      5
  Rejected:       0
  Iteration:      7 / 100
  Confirmed:      0
  Tentative:      5
  Rejected:       0
  Iteration:      8 / 100

```

```

Confirmed:      2
Tentative:      2
Rejected:       1
Iteration:      9 / 100
Confirmed:      2
Tentative:      2
Rejected:       1
Iteration:     10 / 100
Confirmed:      2
Tentative:      2
Rejected:       1
Iteration:     11 / 100
Confirmed:      2
Tentative:      2
Rejected:       1
Iteration:     12 / 100
Confirmed:      2
Tentative:      2
Rejected:       1
Iteration:     13 / 100
Confirmed:      2
Tentative:      2
Rejected:       1
Iteration:     14 / 100
Confirmed:      2
Tentative:      2
Rejected:       1
Iteration:     15 / 100
Confirmed:      2

```

```

boruta_keep = ['Confirmed' if c else
               'Tentative' if t else
               'Reject' for c,t in zip(boruta.support_, boruta.support_weak_)]

```

```

# Combine results and print
boruta_features = zip(data.drop('pcinc', axis=1).columns, boruta.ranking_, boruta_keep)
for feat in boruta_features:
    print('Feature: {:<20} Rank: {}, Keep: {}'.format(feat[0], feat[1], feat[2]))

```

```

Feature: open           Rank: 3,  Keep: Reject
Feature: inf            Rank: 1,  Keep: Confirmed
Feature: land           Rank: 2,  Keep: Reject
Feature: oil            Rank: 1,  Keep: Confirmed
Feature: good           Rank: 1,  Keep: Confirmed

```

```

from google.colab import files
uploaded = files.upload()

```

 No file chosen
 Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving boruta shan.nv to boruta shan (1).nv

```
from boruta_shap import BorutaShap

### Boruta using `BorutaShap`
## Note: takes about 10 minutes

data_X2 = data[['open', 'inf', 'land', 'oil', 'good']]

# Run Boruta algorithm
Feature_Selector = BorutaShap(importance_measure='shap', classification=False)
Feature_Selector.fit(X=data_X2, y=data_Y, n_trials=100, random_state=90095)

# Plot results
Feature_Selector.plot(which_features='all')
plt.show()

# Print results
boruta_keep2 = ['Confirmed' if p in Feature_Selector.accepted else
               'Tentative' if p in Feature_Selector.tentative else
               'Reject' for p in Feature_Selector.all_columns]

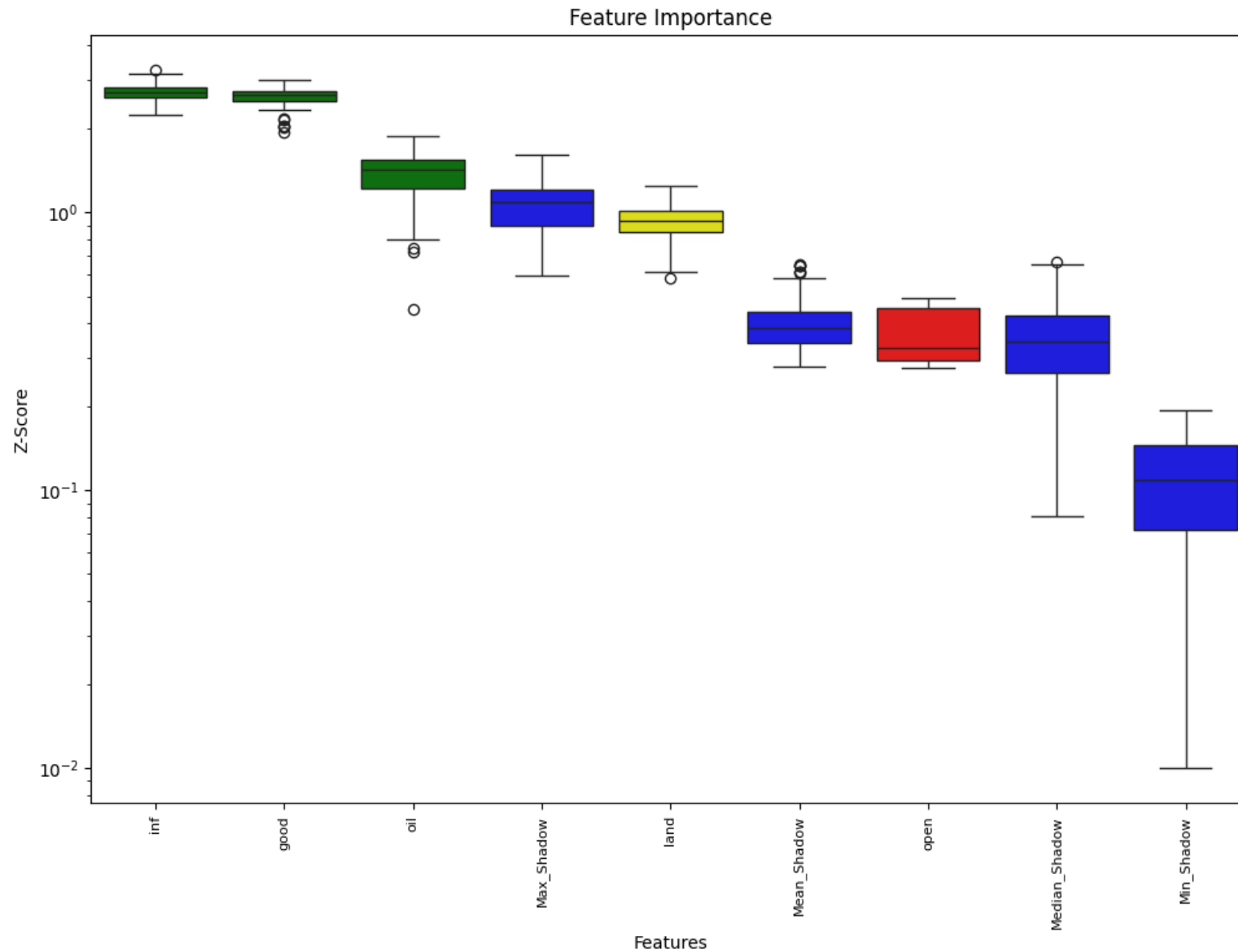
boruta_features2 = zip(Feature_Selector.all_columns,
                      len(Feature_Selector.shap_values) - np.argsort(Feature_Selector.shap_values),
                      Feature_Selector.shap_values,
                      boruta_keep2)
for feat in boruta_features2:
    print('Feature: {:<20} Rank: {} ({:.3f}), Keep: {}'.format(feat[0], feat[1], feat[2], feat[3]))
```




100%

100/100 [00:36<00:00, 2.09it/s]

3 attributes confirmed important: ['oil', 'inf', 'good']
 1 attributes confirmed unimportant: ['open']
 1 tentative attributes remains: ['land']



Feature: open	Rank: 1 (891.059),	Keep: Reject
Feature: inf	Rank: 2 (473.403),	Keep: Confirmed
Feature: land	Rank: 3 (596.391),	Keep: Tentative
Feature: oil	Rank: 7 (1034.957),	Keep: Confirmed
Feature: good	Rank: 4 (480.843),	Keep: Confirmed

The output of Boruta suggests that we should reject openness, tentatively keep land, and confirmed keep inflation, oil, and good. This is interesting, because we already know oil is flawed because of limited observations. Additionally, we know that good is not telling us anything about the country other than the country either provided or made it easy to collect good quality data. Further, Boruta suggesting we reject openness does not bode well for our hypothesis that openness positively influences per capita income. We will continue with Mallows to see how results may differ. Later, we will still try a regression model with log openness, as Boruta does not work with transformed variables, we were not able to use Boruta to evaluate log openness, which may still produce a good, statistically significant model.

```
data1 = data.rename(columns={'inf': 'inflation'})

# First get some estimates from the full regression
data_full = smf.ols(formula='pcinc ~ open + land + oil + good + inflation', data=data1).fit()
data_full_sig2 = data_full.mse_resid # residual MSE = sigma^2 hat
data_n, _ = data1.shape

def negative_mallows(estimator, X, y):
    y_pred = estimator.predict(X)
    sse = np.sum((y - y_pred) ** 2)
    p = X.shape[1] + 1 #number of features plus intercept term
    cp = sse / data_full_sig2 + 2 * p - data_n
    return -cp

# Prepare data
data_y = data1['lpcinc'].values
data_x = data1.drop(['lpcinc', 'lland', 'lopen', 'linfdec', 'opendec', 'pcinc', 'linf'], axis =1)

#linear regression for feature selection
lr = LinearRegression(fit_intercept=True)

data_efs = EFS(lr,
               min_features=1,
               max_features=5,
               scoring=negative_mallows,
               cv=None)

# Run regressions
data_efs.fit(data_x, data_y)
print('Best subset:', [data_efs.feature_names[i] for i in data_efs.best_idx])

🔗 Features: 31/31Best subset: ['good']
```

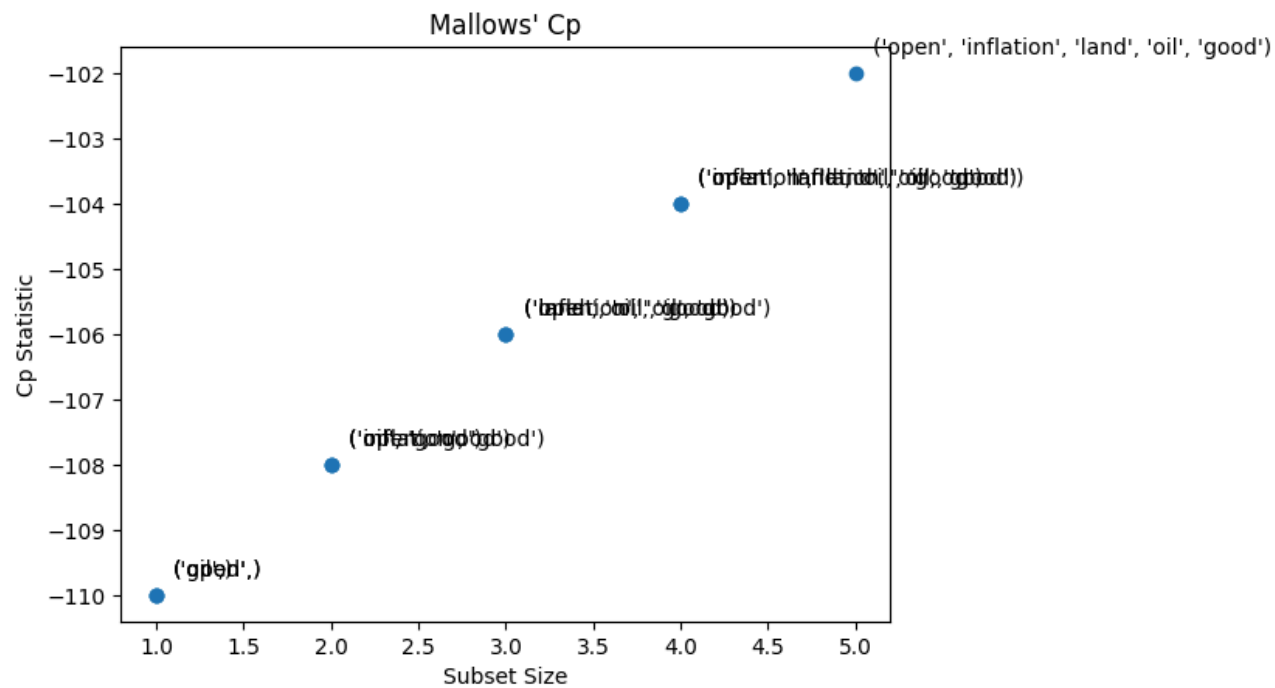
The above code takes the 5 predictor variables, finds 63 different possible regression combinations, and tells us that the best single predictor is good data. This is somewhat concerning, as good data is a dummy variable that =1 if a given observation is high quality data. We will look deeper into this with the full results summary table result following the next code block and the following second attempt at Mallows using a different subset of our data.

```
# Extract Mallows' Cp data
data_efs_features = [data_efs.subsets_[p]['feature_names'] for p in data_efs.subsets_]
data_efs_subset = [len(data_efs.subsets_[p]['feature_idx']) for p in data_efs.subsets_]
data_efs_cp = [-data_efs.subsets_[p]['avg_score'] for p in data_efs.subsets_]
data_mallows = pd.DataFrame({'subsets':data_efs_subset, 'Cp':data_efs_cp}, index=data_efs_features)

# Top 3 best, by subset
data_mallows_best = data_mallows.groupby('subsets')[['subsets','Cp']].apply(lambda x: x.nsmallest(3, 'Cp'))
data_mallows_best

# Plotting best subsets
plt.scatter(data_mallows_best['subsets'], data_mallows_best['Cp'])
plt.title("Mallows' Cp")
plt.xlabel('Subset Size')
plt.ylabel('Cp Statistic')
for i,sub in enumerate(data_mallows_best['subsets']):
    subset = data_mallows_best.index[i][1]
    plt.annotate(subset, (sub + 0.1, data_mallows_best.iloc[i]['Cp'] + 0.3)) # to plot labels
plt.show()

# And the associated sorted data frame
data_mallows_best.sort_values('Cp')
```



subsets			Cp
1	(good,)	1	-109.999989
	(oil,)	1	-109.999988
	(open,)	1	-109.999987
2	(oil, good)	2	-107.999992
	(open, good)	2	-107.999990
	(inflation, good)	2	-107.999990
3	(open, oil, good)	3	-105.999993
	(inflation, oil, good)	3	-105.999992
	(land, oil, good)	3	-105.999992
4	(open, land, oil, good)	4	-103.999993
	(open, inflation, oil, good)	4	-103.999993
	(inflation, land, oil, good)	4	-103.999992
5	(open, inflation, land, oil, good)	5	-101.999993

The above summary table identifies the best three models (all equally good with = Cp scores of -110) as single predictor models using predictor variables good data, oil, and opendec. As mentioned above, single predictor models on our dummy variables will not work well given what we already know about the underlying data. Only seven observations are major oil producers (oil = 1), which means the dataset is too skewed on that variable toward non-major oil producers to use that as our predictor variable. Further, good data is a dummy variable for quality of data, and will therefore also not work well in practice. Below, we re-run Mallows CP on a smaller subset of data that excludes oil and good dummy variables.

```
# First get some estimates from the full regression
data_fullB = smf.ols(formula='pcinc ~ open + land + inflation', data=data1).fit()
data_full_sig2B = data_fullB.mse_resid # residual MSE = sigma^2 hat
data_n, _ = data1.shape

def negative_mallows(estimator, X, y):
    y_pred = estimator.predict(X)
    sse = np.sum((y - y_pred) ** 2)
    p = X.shape[1] + 1 #number of features plus intercept term
    cp = sse / data_full_sig2B + 2 * p - data_n
    return -cp

# Prepare data
data_Y = data1['lpcinc'].values
data_X = data1.drop(['lpcinc', 'lland', 'lopen', 'linfdec', 'pcinc', 'oil', 'opendec', 'linf'], axis =1)

#linear regression for feature selection
lrB = LinearRegression(fit_intercept=True)

data_efsB = EFS(lrB,
                min_features=1,
                max_features=3,
                scoring=negative_mallows,
                cv=None)

# Run regressions
data_efsB.fit(data_X, data_Y)
print('Best subset:', [data_efsB.feature_names[i] for i in data_efsB.best_idx_])

🔗 Features: 14/14Best subset: ['good']
```

The above code re-runs Mallows CP on our three predictor values that are non-dummy variables. With only three variables, there are 7 possible regressions. Mallows CP identifies the strongest option as a single predictor model using openness as the predictor variable.

```
# Extract Mallows' Cp data
data_efsB_features = [data_efsB.subsets_[p]['feature_names'] for p in data_efsB.subsets_]
data_efsB_subset = [len(data_efsB.subsets_[p]['feature_idx']) for p in data_efsB.subsets_]
data_efsB_cp = [-data_efsB.subsets_[p]['avg_score'] for p in data_efsB.subsets_]

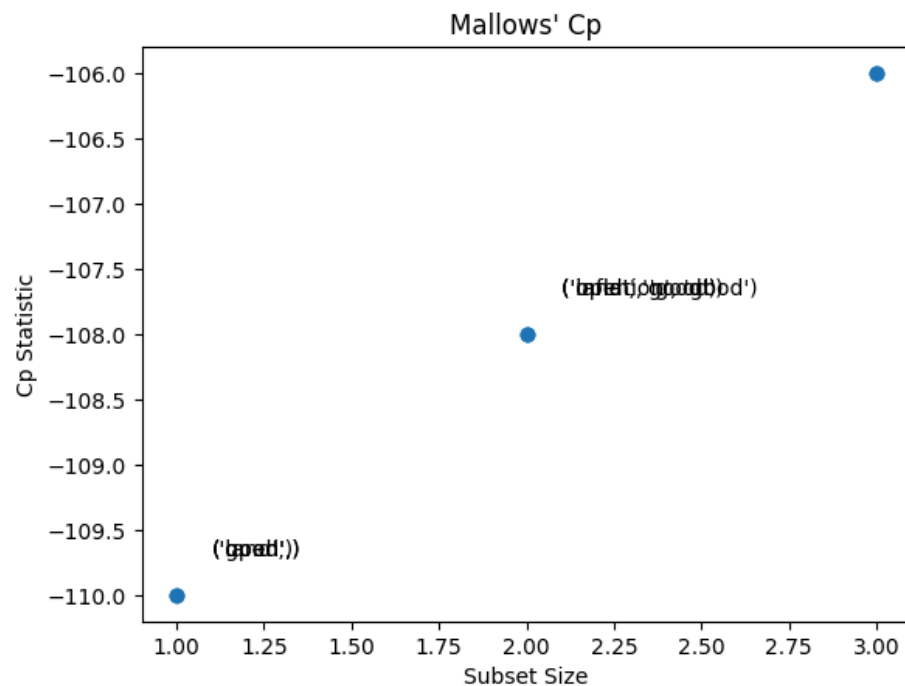
```

```
data_mallowsB = pd.DataFrame({'subsets':data_efsB_subset, 'Cp':data_efsB_cp}, index=data_efsB_features)

# Top 3 best, by subset
data_mallows_bestB = data_mallowsB.groupby('subsets')[['subsets','Cp']].apply(lambda x: x.nsmallest(3, 'Cp'))
data_mallows_bestB

# Plotting best subsets
plt.scatter(data_mallows_bestB['subsets'], data_mallows_bestB['Cp'])
plt.title("Mallows' Cp")
plt.xlabel('Subset Size')
plt.ylabel('Cp Statistic')
for i,sub in enumerate(data_mallows_bestB['subsets']):
    subset = data_mallows_bestB.index[i][1]
    plt.annotate(subset, (sub + 0.1, data_mallows_bestB.iloc[i]['Cp'] + 0.3)) # to plot labels
plt.show()

# And the associated sorted data frame
data_mallows_bestB.sort_values('Cp')
```



		subsets	Cp
subsets			
1	(good,)	1	-109.999993
	(open,)	1	-109.999992
	(land,)	1	-109.999992
2	(open, good)	2	-107.999994
	(inflation, good)	2	-107.999993
	(land, good)	2	-107.999993
3	(open, land, good)	3	-105.999994
	(open, inflation, good)	3	-105.999994
	(inflation, land, good)	3	-105.999993

The subset table illustrates that open, land, and inflation are each equally good single predictor models. Given our analysis of variables in part 1, we know that even though Mallows doesn't work on transformed variables, log transformations of open, land, and inflation will be the OLS models we try and compare given Mallows results.

Interestingly, Mallows illustrates that the weakest model using this subset of the data would be a multi-variable regression using all three variables.

QUESTION 3

Model Building

✓ REGRESSION 1

$$GOOD = \begin{cases} 1, & \text{if data is good} \\ 0, & \text{if data is not good} \end{cases}$$

$$PRICE = \beta_1 + \beta_2 GOOD + e$$

We're choosing the variable good as a predictor as it performed the best both in Boruta and Cp Mallows. Nevertheless, we

```
model1 = smf.ols(' lpcinc~ good', data=data).fit()
print(model1.summary())

model1_std_resid = model1.get_influence().resid_studentized_internal # standardized residuals
model1_lev = model1.get_influence().hat_matrix_diag # leverage values

# Diagnostic plots
fig2, ax2 = plt.subplots(2, 2)

# Residuals vs Fitted plot
ax2[0,0].scatter(x=model1.fittedvalues, y=model1.resid, s=8)
ax2[0,0].set_title('Residuals vs Fitted')

# Normal Q-Q plot
sm.qqplot(model1_std_resid, line='s', ax=ax2[0,1])
ax2[0,1].set_title('Normal Q-Q')

# Scale-Location Plot
ax2[1,0].scatter(x=model1.fittedvalues, y=np.sqrt(np.abs(model1_std_resid)), s=8) # sqrt(|std. resid|)
ax2[1,0].set_title('Scale-Location Plot')

# Residuals vs Leverage plot
ax2[1,1].scatter(x=model1_lev, y=model1_std_resid, s=5)
ax2[1,1].set_title('Residuals vs Leverage')

plt.tight_layout()
plt.show()
```

#REGRESSION VISUALIZATION


```
plt.figure(figsize=(12,6))
plt.rcParams['figure.dpi']=300
plt.scatter(data['good'], data['lpcinc'], color='blue')
plt.plot(data['good'],model1.fittedvalues, color='red')

plt.title("Log Per Capita Income vs Data Quality (good=1)")
plt.xlabel("Good", fontsize = 12)
plt.ylabel("Log Per Capita Income", fontsize = 12)

plt.plot()
```



OLS Regression Results

```

=====
Dep. Variable:          lpcinc      R-squared:                0.224
Model:                  OLS        Adj. R-squared:             0.217
Method:                 Least Squares  F-statistic:              32.34
Date:                  Fri, 25 Oct 2024  Prob (F-statistic):      1.05e-07
Time:                  21:13:46      Log-Likelihood:          -161.53
No. Observations:      114          AIC:                     327.1
Df Residuals:          112          BIC:                     332.5
Df Model:               1
Covariance Type:       nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	7.0732	0.140	50.658	0.000	6.797	7.350
good	1.0767	0.189	5.687	0.000	0.702	1.452

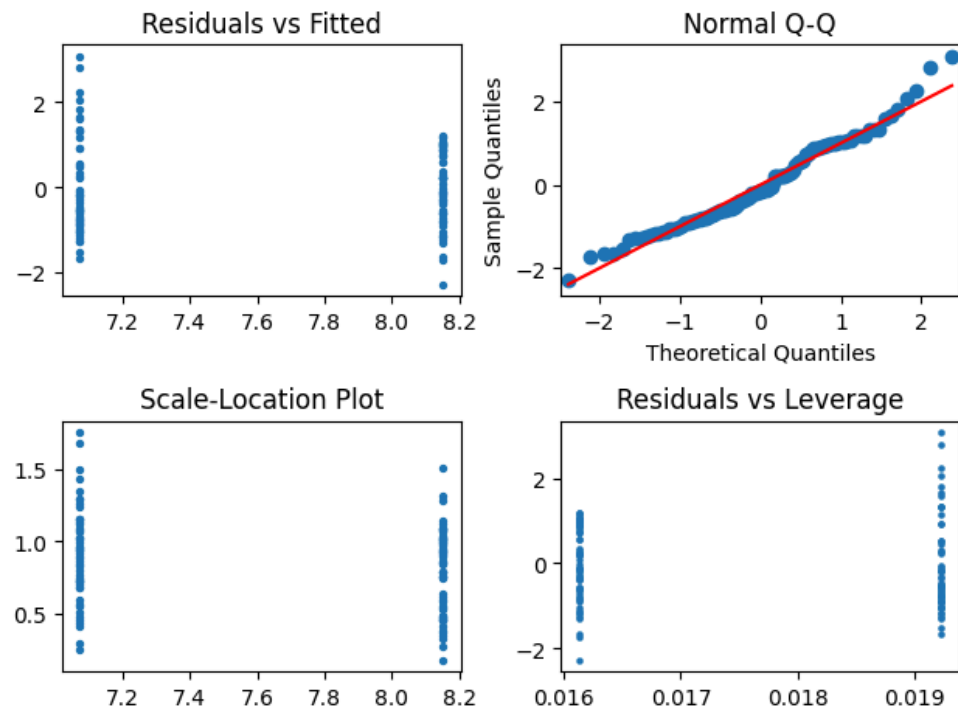
```

=====
Omnibus:                 4.609      Durbin-Watson:           1.973
Prob(Omnibus):           0.100      Jarque-Bera (JB):        4.372
Skew:                    0.480      Prob(JB):                0.112
Kurtosis:                 3.024      Cond. No.:               2.73
=====

```

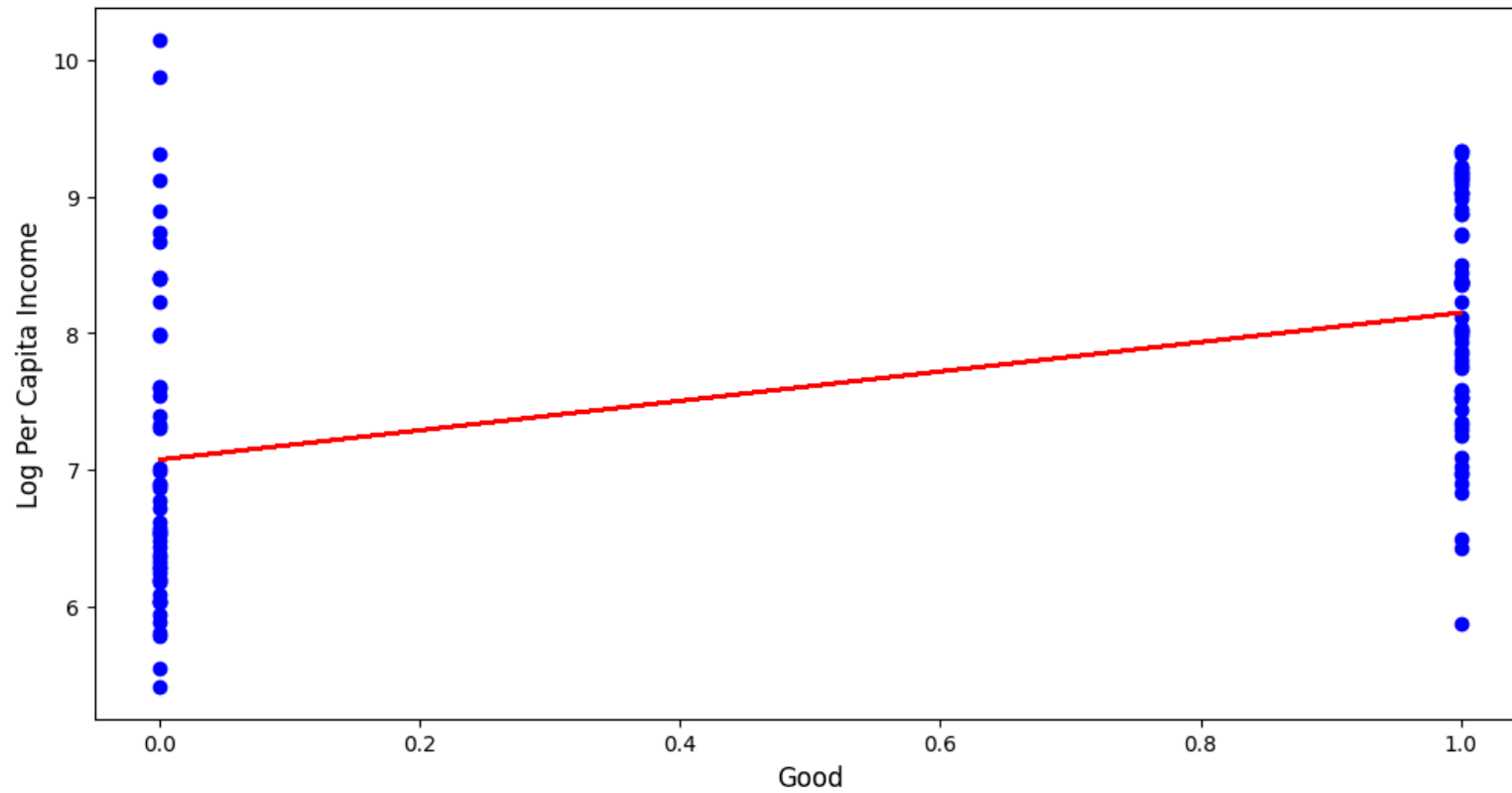
Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.



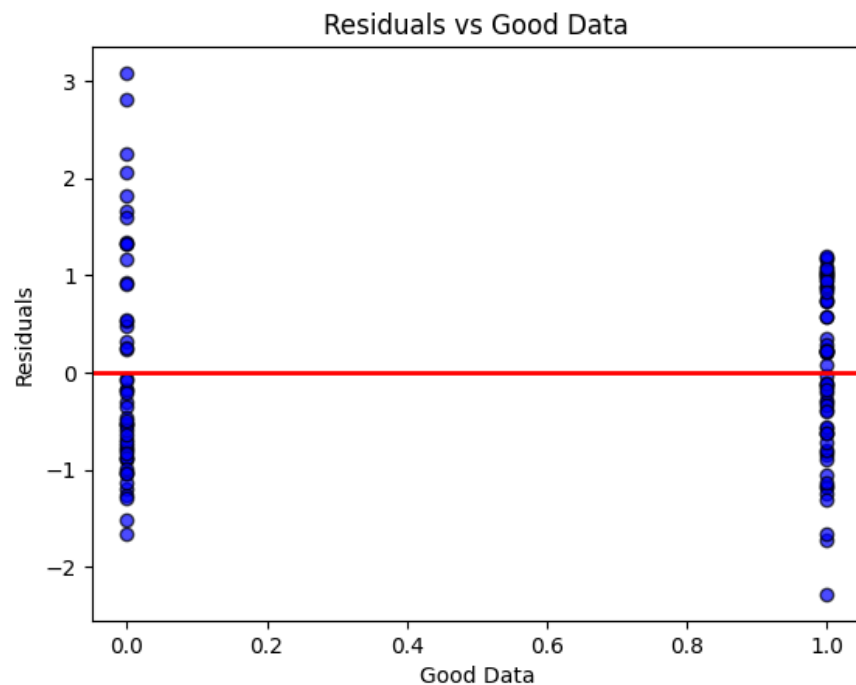
[1]

Log Per Capita Income vs Data Quality (good=1)



Double-click (or enter) to edit

```
plt.scatter(data['good'], model1.resid, color='blue', edgecolor='black', alpha=0.7)
plt.axhline(y=0, color='red', linewidth=2) # Add a horizontal line at y=0
plt.ylabel("Residuals")
plt.xlabel("Good Data")
plt.title("Residuals vs Good Data")
plt.show()
```



```
#cook's
%%R
install.packages("car")
```



WARNING: rpy2.rinterface_lib.callbacks.R[write to console]: Installing package into '/usr/local/lib/R/site-library' (as 'lib' is unspecified)

WARNING: rpy2.rinterface_lib.callbacks.R[write to console]: trying URL 'https://cran.rstudio.com/src/contrib/car_3.1-3.tar.gz'

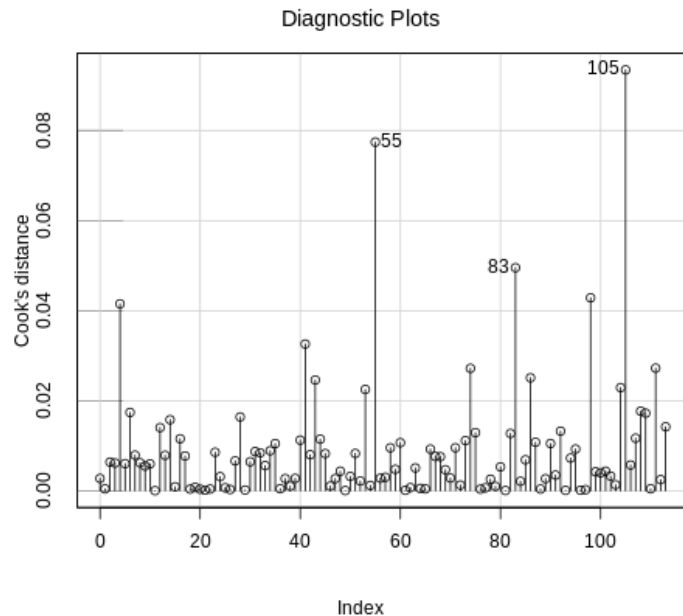
WARNING: rpy2.rinterface_lib.callbacks.R[write to console]: Content type 'application/x-gzip'

WARNING: rpy2.rinterface_lib.callbacks.R[write to console]: length 384407 bytes (375 KB)

WARNING: rpy2.rinterface_lib.callbacks.R[write to console]: =

WARNING: rpy2.rinterface_lib.callbacks.R[write to console]: =

```
%%R
library(car)
mreg.mod <- lm(lpcinc ~ good, data=r_df)
influenceIndexPlot(mreg.mod, id=list(n=3), vars="Cook")
```



```
#Bootstrapping
random.seed(90095)
n_boot = 500 # number of bootstraps

# Initializations
boot_rmse = []
boot_b1 = []
boot_b2 = []
boot_sig2hat = [] # sigma^2 hat estimates (or std error ^2)
boot_r2 = [] # R^2 (no need to adjust)

# Generate a span of x-values (linspace) for plotting
data_span_x = pd.DataFrame({'good': np.linspace(0, 6, 200)}) # Adjust x-range

plt.figure(figsize=(8, 6)) # Increase figure size

for iter in range(n_boot):
    # Sampling with replacement from the original data
    data_sample = data.sample(n=data.shape[0], replace=True) # Sample with replacement
    data_sample_ols = smf.ols('lpcinc ~ good', data=data_sample).fit() # Fit model to sample

    # Append coefficients and metrics
    boot_b1.append(data_sample_ols.params[0]) # Intercept
```

```
boot_b2.append(data_sample_ols.params[1]) # Slope
boot_sig2hat.append(data_sample_ols.scale) # Sigma^2 estimate
boot_rmse.append(np.sqrt(np.mean(data_sample_ols.resid ** 2))) # RMSE
boot_r2.append(data_sample_ols.rsquared) # R^2

# Predict y-values for this bootstrap iteration using data_span_x
data_sample_yhat = data_sample_ols.predict(data_span_x)

# Plot greyed-out bootstrap line with low opacity
plt.plot(data_span_x['good'], data_sample_yhat, color='grey', alpha=0.05)

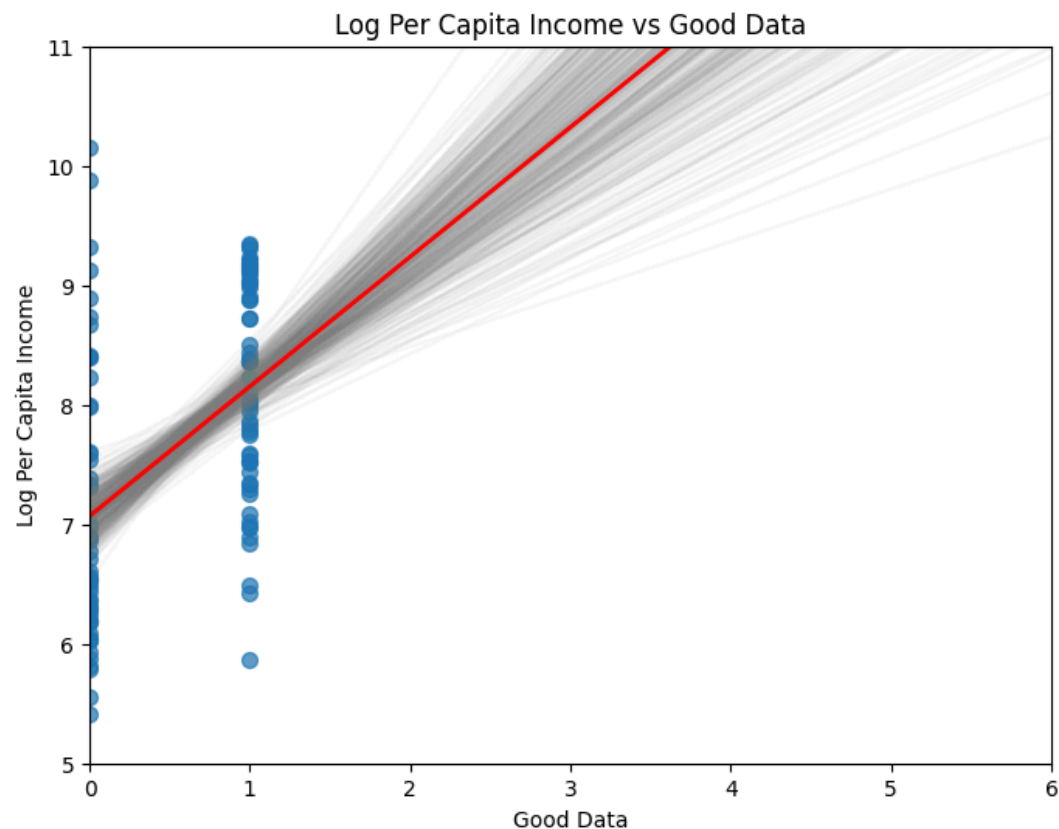
# Fit the averaged model using coefficients from bootstraps
data_ols_span_y = np.mean(boot_b1) + data_span_x['good'] * np.mean(boot_b2)

# Plot the final scatter and averaged fit line
plt.scatter(data['good'], data['lpcinc'], s=50, alpha=0.7) # Scatter with larger points
plt.plot(data_span_x['good'], data_ols_span_y, color='r', linewidth=2) # Averaged red line

# Add labels, title, and grid
plt.xlabel('Good Data')
plt.ylabel('Log Per Capita Income')
plt.title('Log Per Capita Income vs Good Data')

# Adjust x and y axis limits
plt.xlim(0, 6) # Adjust x-axis limit
plt.ylim(5, 11) # Adjust y-axis limit

# Display the plot
plt.show()
```



```

b1_range = np.arange(np.min(boot_b1), np.max(boot_b1), 0.1)
b2_range = np.arange(np.min(boot_b2), np.max(boot_b2), 0.1)

# Plotting beta estimates
fig6, ax6 = plt.subplots(1, 2)
sns.histplot(boot_b1, stat='density', kde=False, ax=ax6[0])
sns.kdeplot(boot_b1, color='fuchsia', ax=ax6[0], label='Kernel Density')
sns.lineplot(x=b1_range, y=norm.pdf(b1_range, np.mean(boot_b1), np.std(boot_b1)), color='lime', ax=ax6[0], label='Normal Density')
ax6[0].set_title('Intercept Dist')
sns.histplot(boot_b2, stat='density', kde=False, ax=ax6[1])
sns.kdeplot(boot_b2, color='fuchsia', ax=ax6[1], label='Kernel Density')
sns.lineplot(x=b2_range, y=norm.pdf(b2_range, np.mean(boot_b2), np.std(boot_b2)), color='lime', ax=ax6[1], label='Normal Density')
ax6[1].set_title('Slope Dist')
plt.tight_layout()
plt.show()

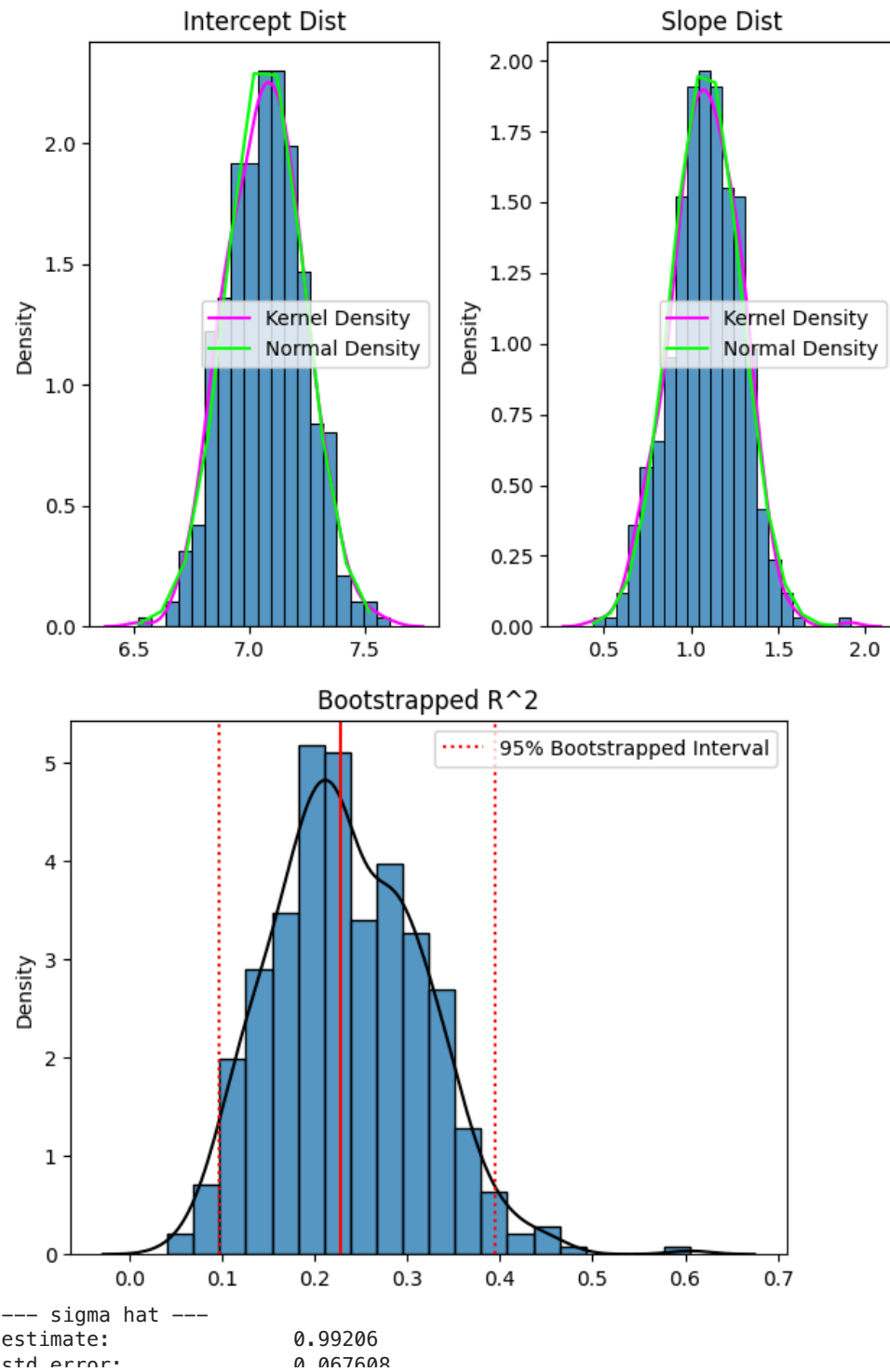
# Bootstrap for distribution of R^2 (coefficient of determination)
sns.histplot(boot_r2, stat='density', kde=False)
sns.kdeplot(boot_r2, color='black')

```



```
plt.axvline(x=np.quantile(boot_r2, q=0.025), color='red', linestyle=':', label='95% Bootstrapped Interval')
plt.axvline(x=np.quantile(boot_r2, q=0.5), color='red')
plt.axvline(x=np.quantile(boot_r2, q=0.975), color='red', linestyle=':')
plt.title('Bootstrapped R^2')
plt.legend()
plt.show()

# Bootstrap for estimated residual standard deviation
boot_stdhat = np.sqrt(boot_sig2hat)
print('--- sigma hat ---\n' +
      f'estimate:\t\t{boot_stdhat.mean():.5}\n' +
      f'std error:\t\t{boot_stdhat.std():.5}\n' +
      f'median:\t\t\t{np.quantile(boot_stdhat, q=0.5):.5}\n' +
      f'2.5 percentile:\t\t{np.quantile(boot_stdhat, q=0.025):.5}\n' +
      f'97.5 percentile:\t{np.quantile(boot_stdhat, q=0.975):.5}'
      )
```



```

5000000
median:      0.99012
2.5 percentile: 0.87059
97.5 percentile: 1.129

```

```

# Cross validation
from sklearn.model_selection import KFold
from sklearn import metrics

good_x = data['good'].to_numpy().reshape(-1,1) # as numpy array

# 5-Fold initialization
kf = KFold(n_splits=5, shuffle=True, random_state=10)

# Initialize lists for appending
fold_assignments = np.zeros(data.shape[0])
rmse_list = []
intercept_list = []
coefficient_list = []

for fold, (train_index, test_index) in enumerate(kf.split(data)):
    fold_assignments[test_index] = fold

    # Split into train and test samples
    X_train, X_test = good_x[train_index], good_x[test_index]
    y_train, y_test = data['lpcinc'][train_index], data['lpcinc'][test_index]

    # Model fitting & evaluation
    model = LinearRegression().fit(X_train, y_train) # train model
    y_pred = model.predict(X_test) # predicted y
    fold_rmse = np.sqrt(metrics.mean_squared_error(y_test, y_pred)) # RMSE

    # Append model info
    rmse_list.append(fold_rmse)
    intercept_list.append(model.intercept_)
    coefficient_list.append(model.coef_[0]) # only one coefficient

# Insert fold assignments
infreg_folds = data.copy()[['good', 'lpcinc']]
infreg_folds['fold'] = fold_assignments

# Plotting
fold_c = ['#440154FF', '#3B528BFF', '#21908CFF', '#5DC863FF', '#FDE725FF']
plt.scatter(x=infreg_folds['good'], y=infreg_folds['lpcinc'], c=infreg_folds['fold'])
for i, params in enumerate(zip(intercept_list, coefficient_list)):
    plt.axline(xy1=(0, params[0]), slope=params[1], color=fold_c[i], label=f'Fold {i+1}')
plt.ylabel('Log Per Capita Income')
plt.xlabel('Good')
plt.legend()
plt.tight_layout()

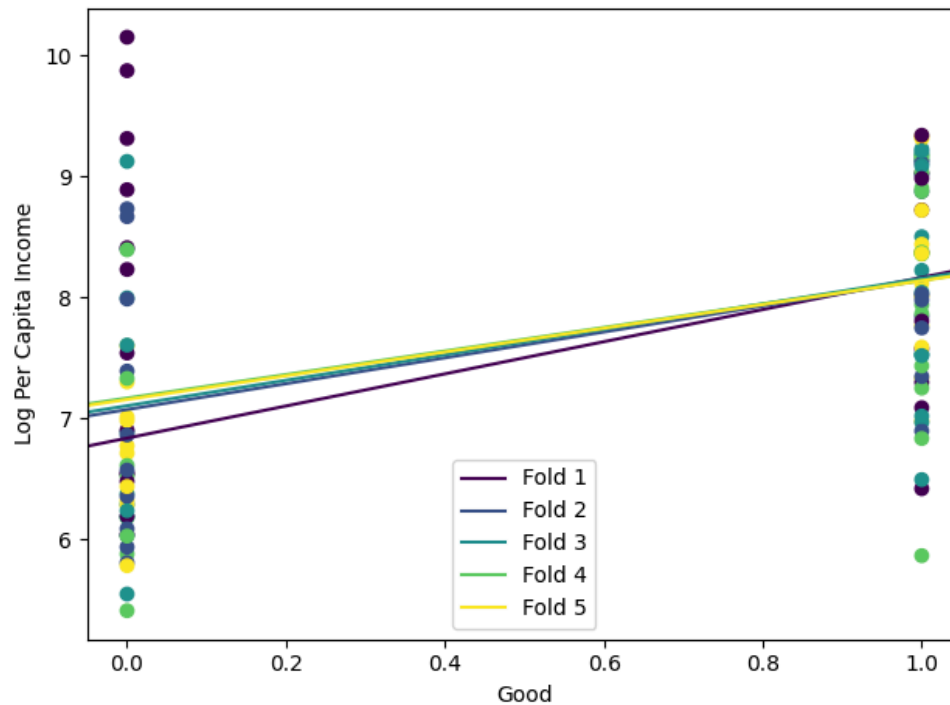
```

```
## Scatter plot has the true values (colored based on respective hold-out fold)
## Lines are the models for each fold
```

```
## We need to check if the model's prediction accuracy isn't varying too much for any one particular sample,
## and if the lines of best fit don't vary too much with respect the the slope and level.
```

```
# CV results
print(f'CV RMSE = {np.mean(rmse_list)}')
```

CV RMSE = 0.9979334917040955



```
+import statsmodels.stats.api as sms
from simple_colors import *
# Linearity: Harvey-Collier --> Ho: model is linear
name = ["t-stat", "p-value"]
test = sms.linear_harvey_collier(model1)
print(blue("Linearity Test Results:", ['bold']))
print(list(zip(name, test)))
print("\n")
```

```
# Normality of the Residuals: Jarque-Bera --> Residuals ~ N(0,1)
name = ["Jarque-Bera", "Chi^2 two-tail prob.", "Skew", "Kurtosis"]
test = sms.jarque_bera(model1.resid)
print(blue("JB Results:", ['bold']))
```

```
print(list(zip(name, test)))
print("\n")
```

```
# Heteroskedasticity: Breush-Pagan --> Ho: var = constant
name = ["Lagrange multiplier statistic", "p-value", "f-value", "f p-value"]
test = sms.het_breuschpagan(model1.resid, model1.model.exog)
print(blue("BP Results:", ['bold']))
print(list(zip(name, test)))
```

```
↔ Linearity Test Results:
[('t-stat', -0.8130112503808196), ('p-value', 0.41796780624664454)]
```

```
JB Results:
[('Jarque-Bera', 4.3723923177555735), ('Chi^2 two-tail prob.', 0.11234327284890087), ('Skew', 0.47956102908865583), ('Kurtosis', 3.02425950042)
```

```
BP Results:
[('Lagrange multiplier statistic', 4.656839580682055), ('p-value', 0.030930073180672184), ('f-value', 4.769992297974984), ('f p-value', 0.0310
```

✓ RERESSION 2: LOG INFLATION

$$PCINC = \beta_1 + \beta_2 LINF + e$$

```
#Regression with log inflation
model2 = smf.ols(' lpcinc~ linf', data=data).fit()
print(model2.summary())
```

```
model2_std_resid = model2.get_influence().resid_studentized_internal # standardized residuals
model2_lev = model2.get_influence().hat_matrix_diag # leverage values
```

```
# Diagnostic plots
fig2, ax2 = plt.subplots(2, 2, figsize=(12, 8))
```

```
# Residuals vs Fitted plot
ax2[0,0].scatter(x=model2.fittedvalues, y=model2.resid, s=8)
ax2[0,0].set_title('Residuals vs Fitted')
```

```
# Normal Q-Q plot
sm.qqplot(model2_std_resid, line='s', ax=ax2[0,1])
ax2[0,1].set_title('Normal Q-Q')
```

```
# Scale-Location Plot
ax2[1,0].scatter(x=model2.fittedvalues, y=np.sqrt(np.abs(model2_std_resid)), s=8) # sqrt(|std. resid|)
ax2[1,0].set_title('Scale-Location Plot')
```

```
# Residuals vs Leverage plot
ax2[1,1].scatter(x=model2_lev, y=model2_std_resid, s=8)
```

```
ax2[1,1].scatter(x=model2_lev, y=model2_std_resid, s=3)  
ax2[1,1].set_title('Residuals vs Leverage')
```

```
plt.tight_layout()  
plt.show()
```

```
#REGRESSION VISUALIZATION
```

```
plt.figure(figsize=(12,6))  
plt.rcParams['figure.dpi']=100  
plt.scatter(data['linf'], data['lpcinc'], color='blue')  
plt.plot(data['linf'],model2.fittedvalues, color='red')
```

```
plt.title("Log Per Capita Income vs Log Inflation")  
plt.xlabel("Log Inflation", fontsize = 12)  
plt.ylabel("Log Per Capita Income", fontsize = 12)
```

```
plt.plot()
```



OLS Regression Results

```

=====
Dep. Variable:          lpcinc    R-squared:                0.022
Model:                  OLS       Adj. R-squared:           0.013
Method:                 Least Squares   F-statistic:             2.542
Date:                   Fri, 25 Oct 2024   Prob (F-statistic):      0.114
Time:                   21:27:43    Log-Likelihood:         -174.71
No. Observations:       114         AIC:                    353.4
Df Residuals:           112         BIC:                    358.9
Df Model:                1
Covariance Type:        nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	8.2569	0.390	21.185	0.000	7.485	9.029
linf	-0.2390	0.150	-1.594	0.114	-0.536	0.058

```

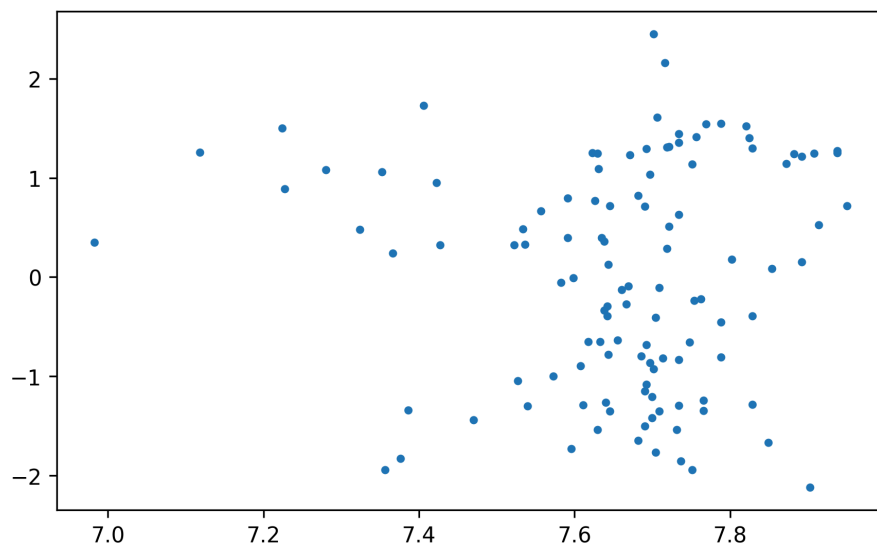
=====
Omnibus:                 35.034    Durbin-Watson:           2.005
Prob(Omnibus):            0.000    Jarque-Bera (JB):        6.538
Skew:                     -0.059    Prob(JB):                0.0380
Kurtosis:                 1.833    Cond. No.                 10.9
=====

```

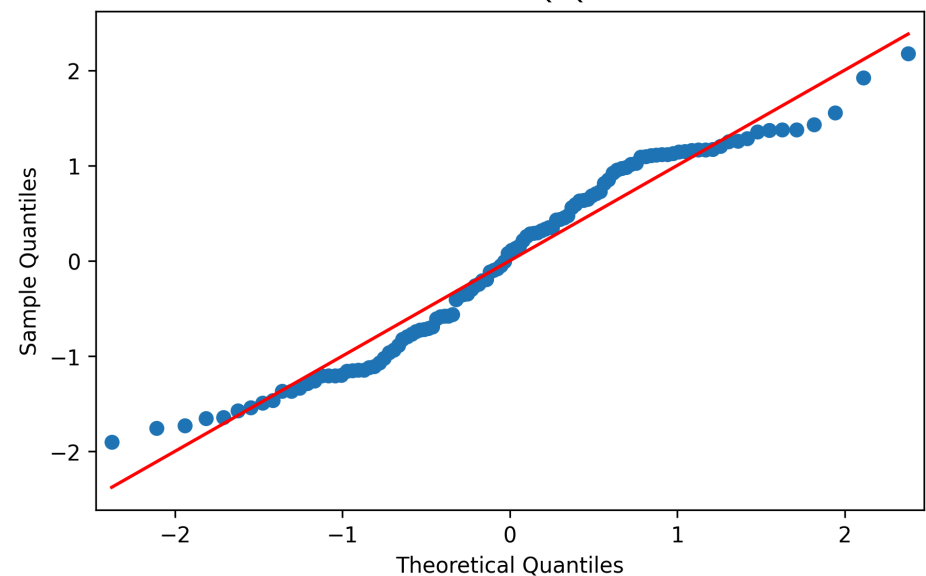
Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

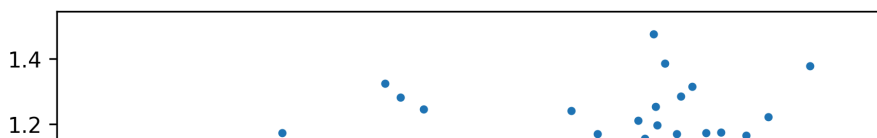
Residuals vs Fitted



Normal Q-Q

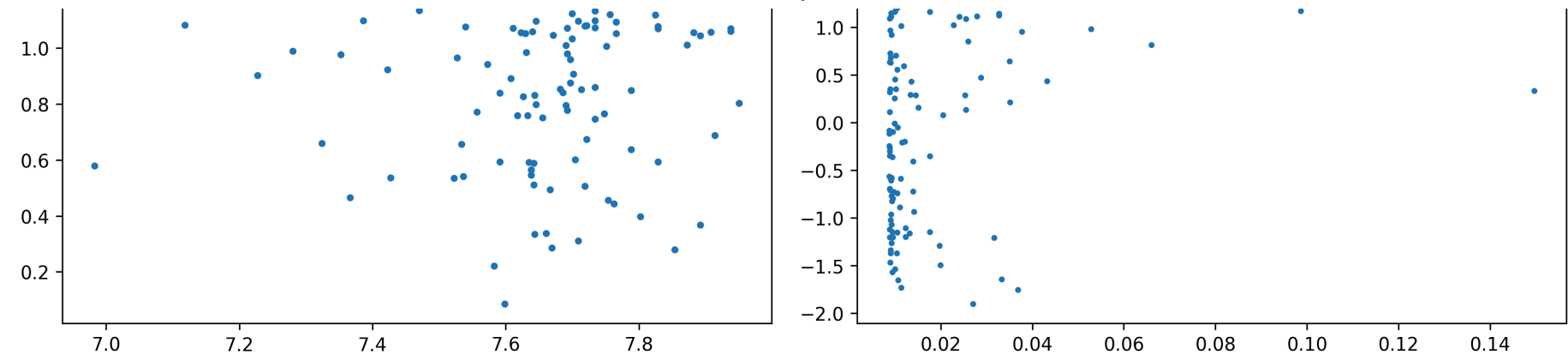


Scale-Location Plot



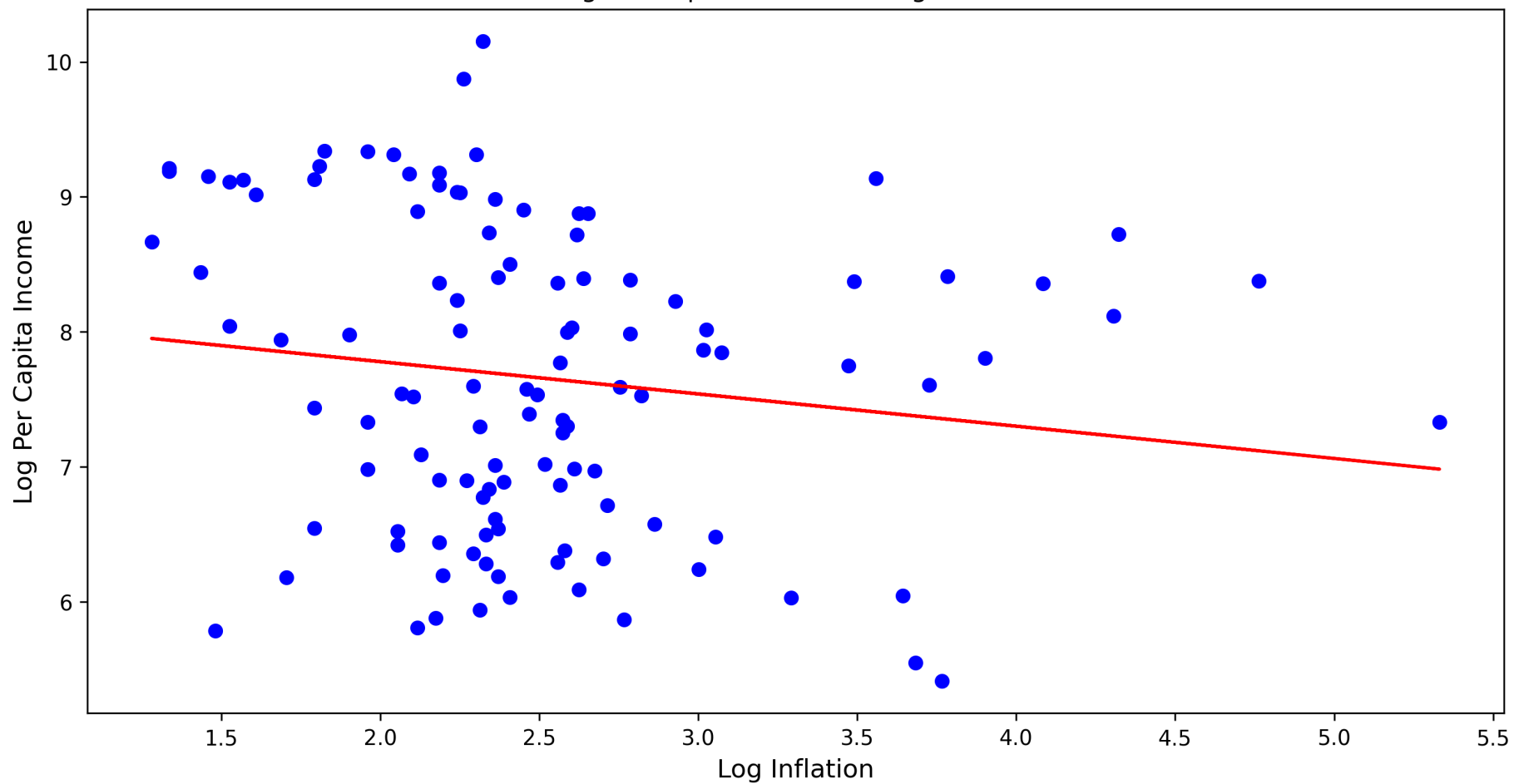
Residuals vs Leverage



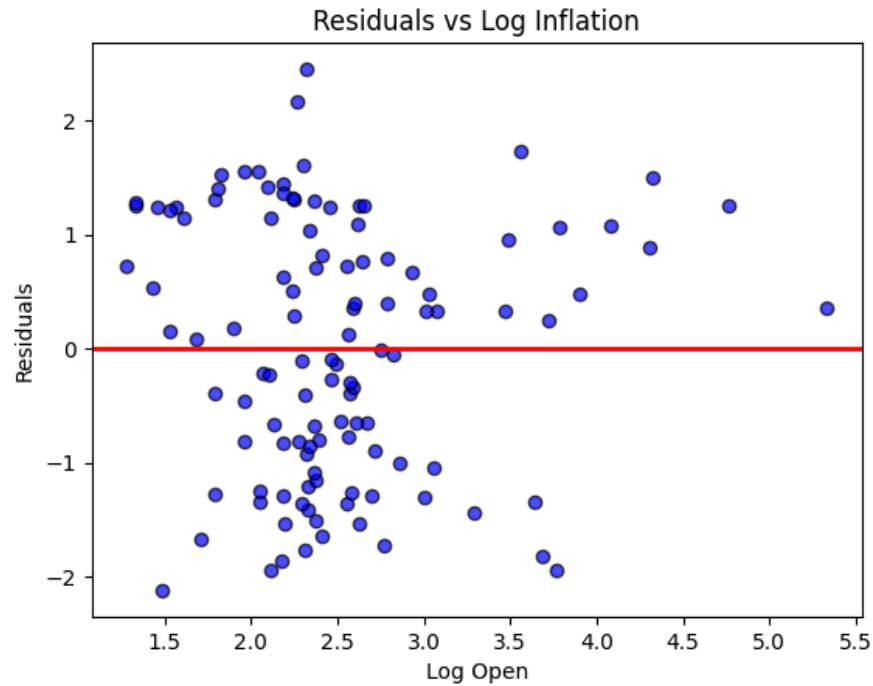


[1]

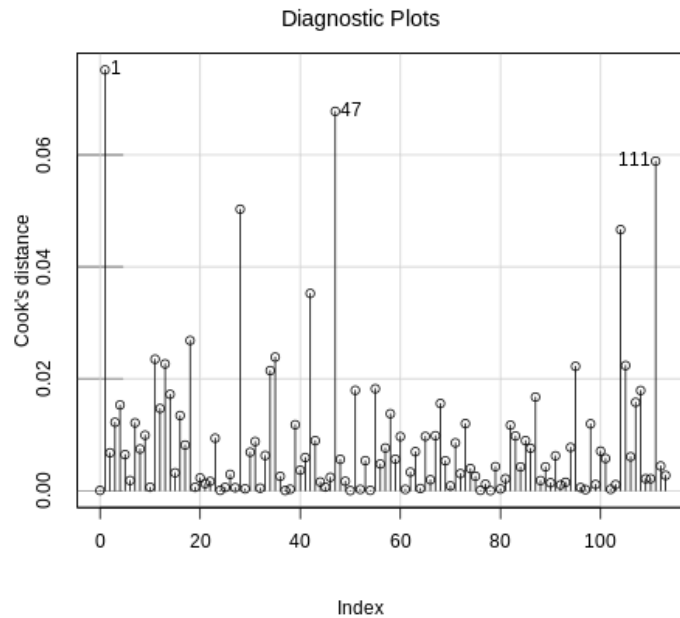
Log Per Capita Income vs Log Inflation




```
plt.scatter(data['linf'], model2.resid, color='blue', edgecolor='black', alpha=0.7)
plt.axhline(y=0, color='red', linewidth=2) # Add a horizontal line at y=0
plt.ylabel("Residuals")
plt.xlabel("Log Open")
plt.title("Residuals vs Log Inflation")
plt.show()
```



```
%%R
library(car)
mreg.mod <- lm(lpcinc ~ linf, data=r_df)
influenceIndexPlot(mreg.mod, id=list(n=3), vars="Cook")
```



```
#Bootstrapping
random.seed(90095)
n_boot = 500 # number of bootstraps

# Initializations
boot_rmse = []
boot_b1 = []
boot_b2 = []
boot_sig2hat = [] # sigma^2 hat estimates (or std error ^2)
boot_r2 = [] # R^2 (no need to adjust)

# Generate a span of x-values (linf) for plotting
data_span_x = pd.DataFrame({'linf': np.linspace(0, 6, 200)}) # Adjust x-range

plt.figure(figsize=(8, 6)) # Increase figure size

for iter in range(n_boot):
    # Sampling with replacement from the original data
    data_sample = data.sample(n=data.shape[0], replace=True) # Sample with replacement
    data_sample_ols = smf.ols('lpcinc ~ linf', data=data_sample).fit() # Fit model to sample

    # Append coefficients and metrics
    boot_b1.append(data_sample_ols.params[0]) # Intercept
```

```
boot_b2.append(data_sample_ols.params[1]) # Slope
boot_sig2hat.append(data_sample_ols.scale) # Sigma^2 estimate
boot_rmse.append(np.sqrt(np.mean(data_sample_ols.resid ** 2))) # RMSE
boot_r2.append(data_sample_ols.rsquared) # R^2

# Predict y-values for this bootstrap iteration using data_span_x
data_sample_yhat = data_sample_ols.predict(data_span_x)

# Plot greyed-out bootstrap line with low opacity
plt.plot(data_span_x['linf'], data_sample_yhat, color='grey', alpha=0.05)

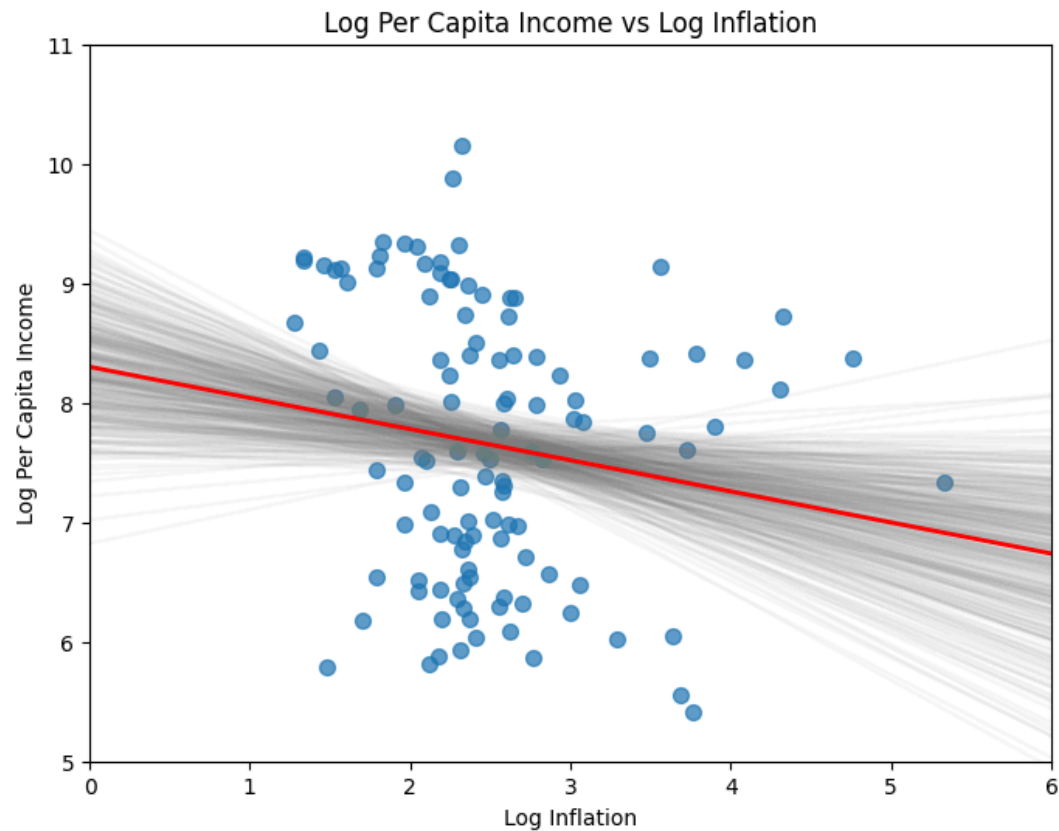
# Fit the averaged model using coefficients from bootstraps
data_ols_span_y = np.mean(boot_b1) + data_span_x['linf'] * np.mean(boot_b2)

# Plot the final scatter and averaged fit line
plt.scatter(data['linf'], data['lpcinc'], s=50, alpha=0.7) # Scatter with larger points
plt.plot(data_span_x['linf'], data_ols_span_y, color='r', linewidth=2) # Averaged red line

# Add labels, title, and grid
plt.xlabel('Log Inflation')
plt.ylabel('Log Per Capita Income')
plt.title('Log Per Capita Income vs Log Inflation')

# Adjust x and y axis limits
plt.xlim(0, 6) # Adjust x-axis limit
plt.ylim(5, 11) # Adjust y-axis limit

# Display the plot
plt.show()
```



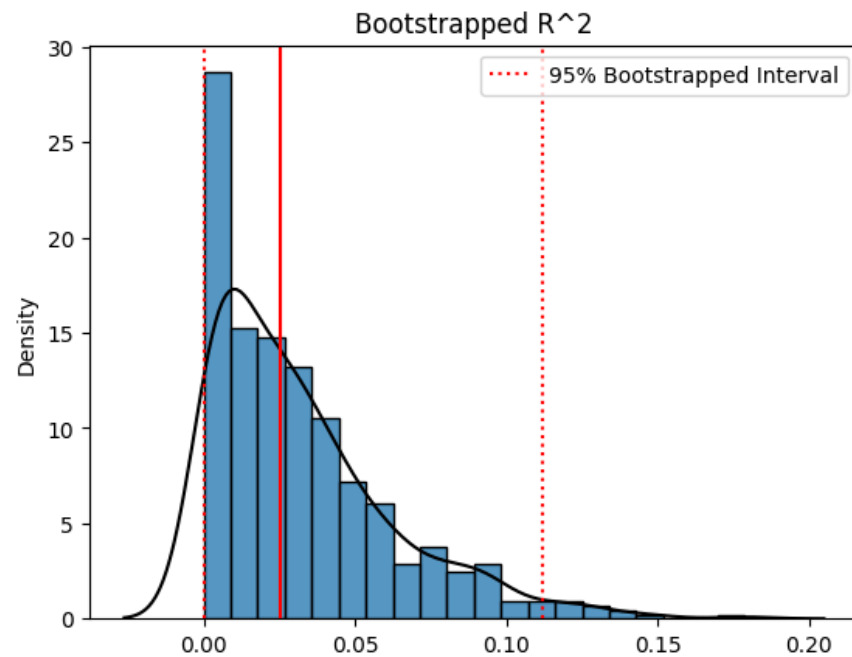
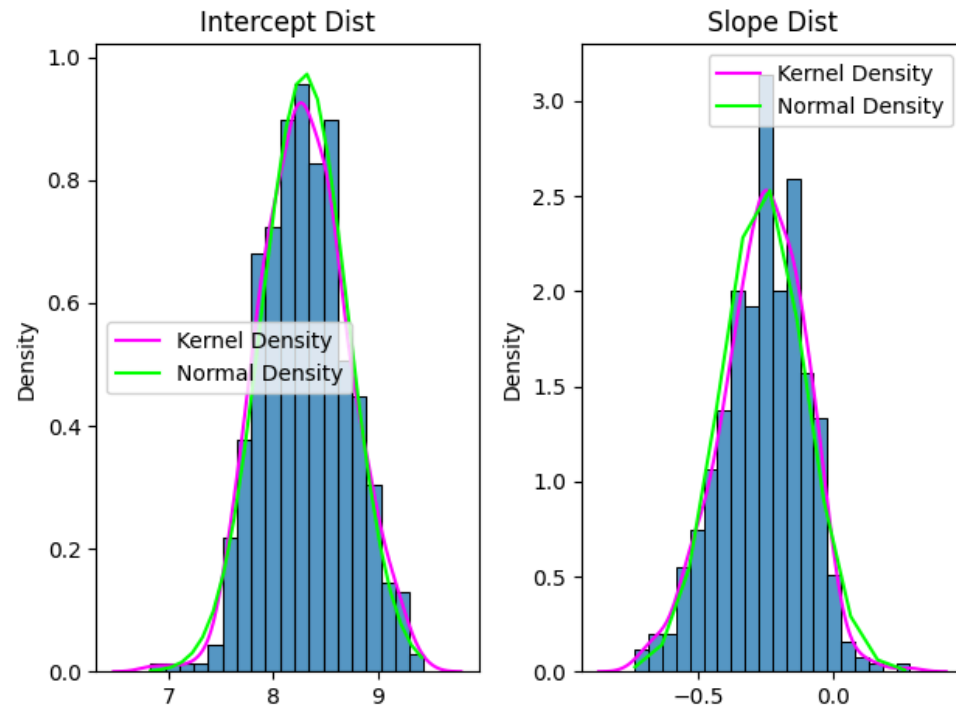
```
b1_range = np.arange(np.min(boot_b1), np.max(boot_b1), 0.1)
b2_range = np.arange(np.min(boot_b2), np.max(boot_b2), 0.1)

# Plotting beta estimates
fig6, ax6 = plt.subplots(1, 2)
sns.histplot(boot_b1, stat='density', kde=False, ax=ax6[0])
sns.kdeplot(boot_b1, color='fuchsia', ax=ax6[0], label='Kernel Density')
sns.lineplot(x=b1_range, y=norm.pdf(b1_range, np.mean(boot_b1), np.std(boot_b1)), color='lime', ax=ax6[0], label='Normal Density')
ax6[0].set_title('Intercept Dist')
sns.histplot(boot_b2, stat='density', kde=False, ax=ax6[1])
sns.kdeplot(boot_b2, color='fuchsia', ax=ax6[1], label='Kernel Density')
sns.lineplot(x=b2_range, y=norm.pdf(b2_range, np.mean(boot_b2), np.std(boot_b2)), color='lime', ax=ax6[1], label='Normal Density')
ax6[1].set_title('Slope Dist')
plt.tight_layout()
plt.show()

# Bootstrap for distribution of R^2 (coefficient of determination)
sns.histplot(boot_r2, stat='density', kde=False)
sns.kdeplot(boot_r2, color='black')
```

```
plt.axvline(x=np.quantile(boot_r2, q=0.025), color='red', linestyle=':', label='95% Bootstrapped Interval')
plt.axvline(x=np.quantile(boot_r2, q=0.5), color='red')
plt.axvline(x=np.quantile(boot_r2, q=0.975), color='red', linestyle=':')
plt.title('Bootstrapped R^2')
plt.legend()
plt.show()

# Bootstrap for estimated residual standard deviation
boot_stdhat = np.sqrt(boot_sig2hat)
print('--- sigma hat ---\n' +
      f'estimate:\t\t{boot_stdhat.mean():.5}\n' +
      f'std error:\t\t{boot_stdhat.std():.5}\n' +
      f'median:\t\t\t{np.quantile(boot_stdhat, q=0.5):.5}\n' +
      f'2.5 percentile:\t\t{np.quantile(boot_stdhat, q=0.025):.5}\n' +
      f'97.5 percentile:\t{np.quantile(boot_stdhat, q=0.975):.5}'
      )
```



--- sigma hat ---

estimate: 1.1171
std error: 0.010780

```

summary
median:      1.1157
2.5 percentile: 1.0125
97.5 percentile: 1.211

```

```

# Cross validation
from sklearn.model_selection import KFold
from sklearn import metrics

inflation_x = data['lnf'].to_numpy().reshape(-1,1) # as numpy array

# 5-Fold initialization
kf = KFold(n_splits=5, shuffle=True, random_state=10)

# Initialize lists for appending
fold_assignments = np.zeros(data.shape[0])
rmse_list = []
intercept_list = []
coefficient_list = []

for fold, (train_index, test_index) in enumerate(kf.split(data)):
    fold_assignments[test_index] = fold

    # Split into train and test samples
    X_train, X_test = inflation_x[train_index], inflation_x[test_index]
    y_train, y_test = data['lpcinc'][train_index], data['lpcinc'][test_index]

    # Model fitting & evaluation
    model = LinearRegression().fit(X_train, y_train) # train model
    y_pred = model.predict(X_test) # predicted y
    fold_rmse = np.sqrt(metrics.mean_squared_error(y_test, y_pred)) # RMSE

    # Append model info
    rmse_list.append(fold_rmse)
    intercept_list.append(model.intercept_)
    coefficient_list.append(model.coef_[0]) # only one coefficient

# Insert fold assignments
infreg_folds = data.copy()[['lnf', 'lpcinc']]
infreg_folds['fold'] = fold_assignments

# Plotting
fold_c = ['#440154FF', '#3B528BFF', '#21908CFF', '#5DC863FF', '#FDE725FF']
plt.scatter(x=infreg_folds['lnf'], y=infreg_folds['lpcinc'], c=infreg_folds['fold'])
for i, params in enumerate(zip(intercept_list, coefficient_list)):
    plt.axline(xy1=(0, params[0]), slope=params[1], color=fold_c[i], label=f'Fold {i+1}')
plt.ylabel('Log Per Capita Income')
plt.xlabel('Log Inflation')
plt.legend()
plt.tight_layout()

```

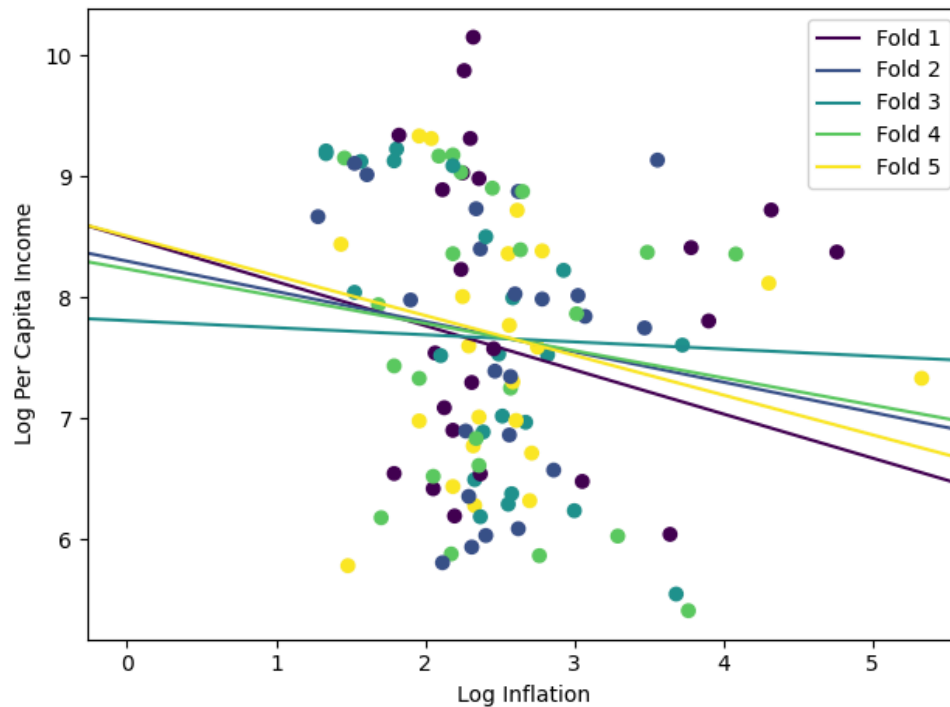


```
## Scatter plot has the true values (colored based on respective hold-out fold)
## Lines are the models for each fold
```

```
## We need to check if the model's prediction accuracy isn't varying too much for any one particular sample,
## and if the lines of best fit don't vary too much with respect the the slope and level.
```

```
# CV results
print(f'CV RMSE = {np.mean(rmse_list)}')
```

```
CV RMSE = 1.1419914411272685
```



Start coding or [generate](#) with AI.

```
import statsmodels.stats.api as sms
from simple_colors import *
# Linearity: Harvey-Collier --> Ho: model is linear
name = ["t-stat", "p-value"]
test = sms.linear_harvey_collier(model2)
print(blue("Linearity Test Results:", ['bold']))
print(list(zip(name, test)))
print("\n")
```

```
# Normality of the Residuals: Jarque-Bera --> Residuals ~ N(0,1)
name = ["Jarque-Bera", "Chi-Sq. df", "p-value", "Skew", "Kurtosis"]
```

```
name = ['Jarque-Bera', 'Chi^2 two-tail prob.', 'Skew', 'Kurtosis']
test = sms.jarque_bera(model2.resid)
print(blue("JB Results:", ['bold']))
print(list(zip(name, test)))
print("\n")
```

```
# Heteroskedasticity: Breush-Pagan --> Ho: var = constant
name = ["Lagrange multiplier statistic", "p-value", "f-value", "f p-value"]
test = sms.het_breuschpagan(model2.resid, model2.model.exog)
print(blue("BP Results:", ['bold']))
print(list(zip(name, test)))
```

➡ **Linearity Test Results:**
 [('t-stat', -0.6496524359830801), ('p-value', 0.5172710778262974)]

JB Results:
 [('Jarque-Bera', 6.538088800125302), ('Chi^2 two-tail prob.', 0.038042763373223175), ('Skew', -0.05884623087459621), ('Kurtosis', 1.83270045321)]

BP Results:
 [('Lagrange multiplier statistic', 0.5460820258457801), ('p-value', 0.4599230100988705), ('f-value', 0.5390839557313489), ('f p-value', 0.46431)]

✓ REGRESSION 3: LOG OPEN

$$PCINC = \beta_1 + \beta_2 LOPEN + e$$

```
model5 = smf.ols(' lpcinc~ lopen', data=data).fit()
print(model5.summary())
```

```
model5_std_resid = model5.get_influence().resid_studentized_internal # standardized residuals
model5_lev = model5.get_influence().hat_matrix_diag # leverage values
```

```
# Diagnostic plots
fig2, ax2 = plt.subplots(2, 2)
```

```
# Residuals vs Fitted plot
ax2[0,0].scatter(x=model5.fittedvalues, y=model5.resid, s=8)
ax2[0,0].set_title('Residuals vs Fitted')
```

```
# Normal Q-Q plot
sm.qqplot(model5_std_resid, line='s', ax=ax2[0,1])
ax2[0,1].set_title('Normal Q-Q')
```

```
# Scale-Location Plot
ax2[1,0].scatter(x=model5.fittedvalues, y=np.sqrt(np.abs(model5_std_resid)), s=8) # sqrt(|std. resid|)
ax2[1,0].set_title('Scale-Location Plot')
```

```
# Residuals vs Leverage plot
ax2[1,1].scatter(x=model5_lev, y=model5_std_resid, s=5)
ax2[1,1].set_title('Residuals vs Leverage')

plt.tight_layout()
plt.show()

#REGRESSION VISUALIZATION
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# Generate finer x-values for a smooth regression line
x = np.linspace(data['lopen'].min(), data['lopen'].max(), 200)

# Predict y-values (lpcinc) based on the model using the generated x-values
y = model5.params[0] + model5.params[1] * x
plt.figure(figsize=(12, 6))
plt.scatter(data['lopen'], data['lpcinc'], color='blue', alpha=0.7, s=50)

plt.plot(x, y, color='red', linewidth=2)

plt.title("Log Per Capita Income vs Log Open")
plt.xlabel("Log Open", fontsize=12)
plt.ylabel("Log Per Capita Income", fontsize=12)

plt.show()
```



OLS Regression Results

```

=====
Dep. Variable:          lpcinc    R-squared:                0.024
Model:                  OLS      Adj. R-squared:            0.015
Method:                 Least Squares    F-statistic:              2.763
Date:                   Fri, 25 Oct 2024    Prob (F-statistic):       0.0992
Time:                   23:12:08    Log-Likelihood:          -174.60
No. Observations:      114      AIC:                     353.2
Df Residuals:          112      BIC:                     358.7
Df Model:               1
Covariance Type:       nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	6.6324	0.626	10.588	0.000	5.391	7.874
lopen	0.2983	0.179	1.662	0.099	-0.057	0.654

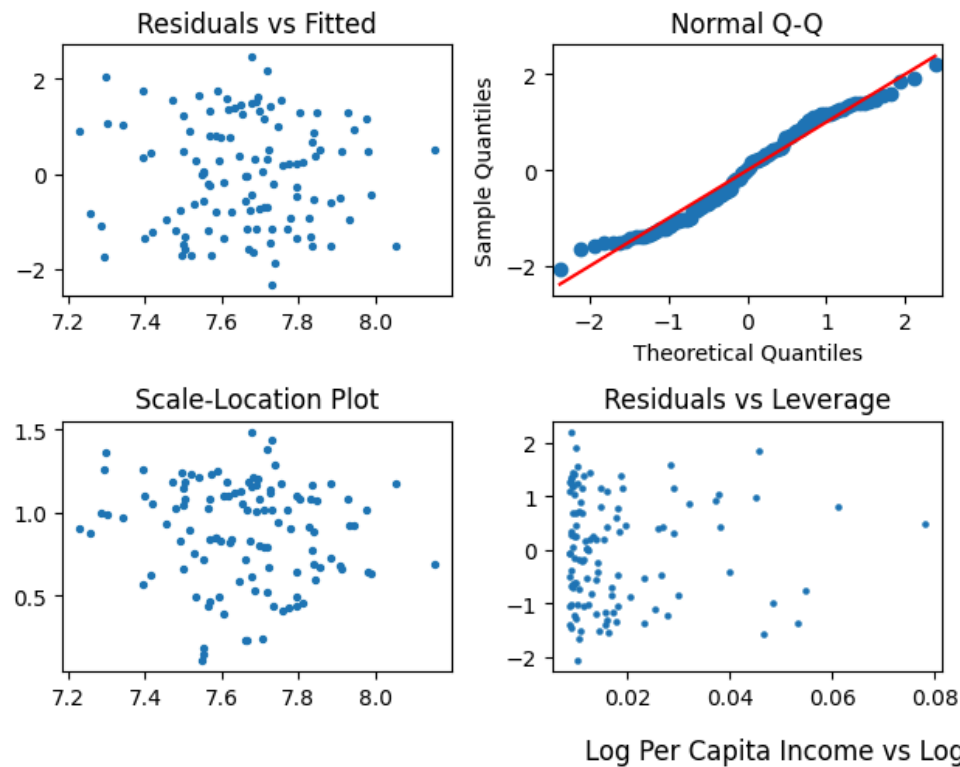
```

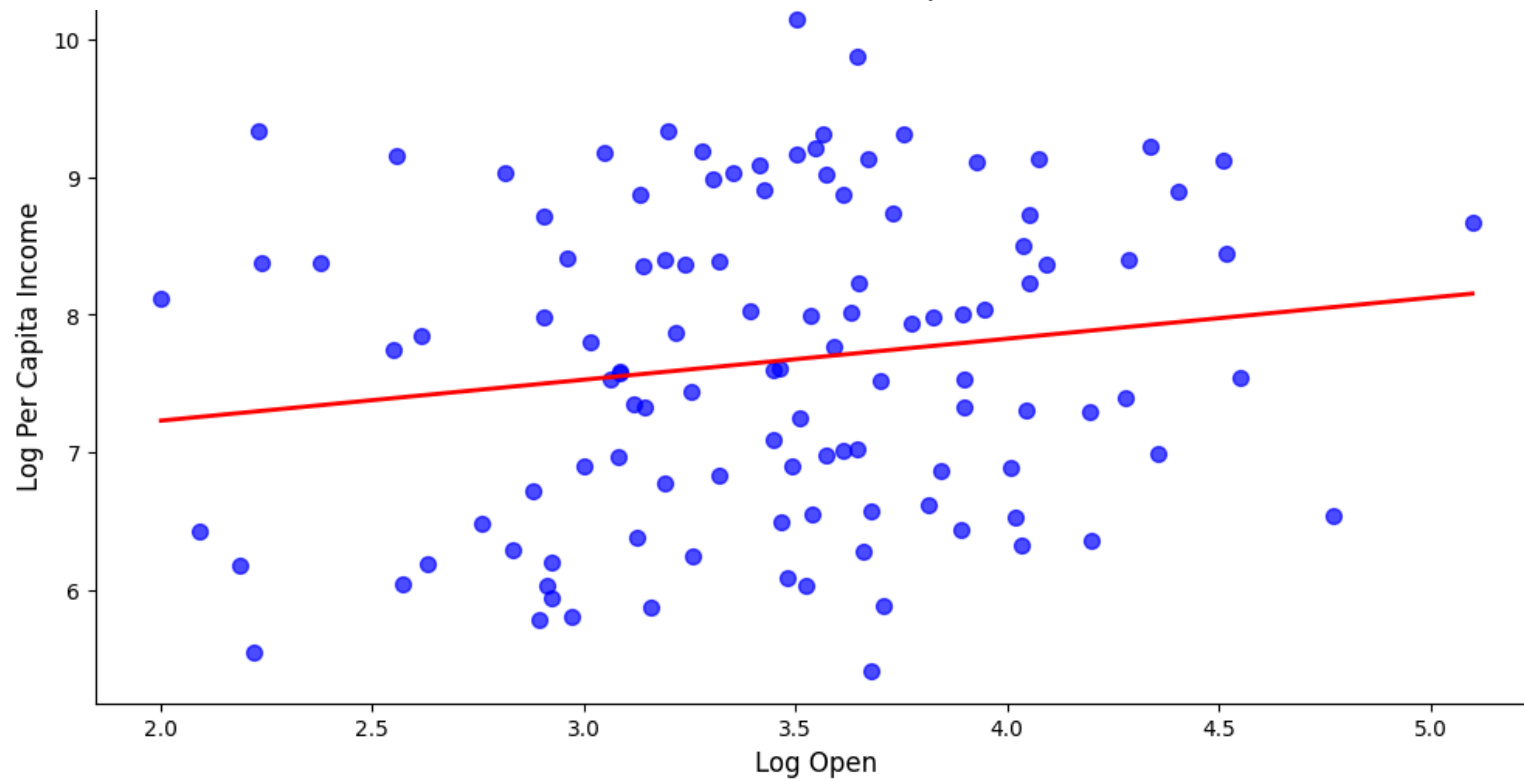
=====
Omnibus:                24.057    Durbin-Watson:           1.987
Prob(Omnibus):           0.000    Jarque-Bera (JB):        5.642
Skew:                    0.055    Prob(JB):                0.0596
Kurtosis:                1.916    Cond. No.:               22.3
=====

```

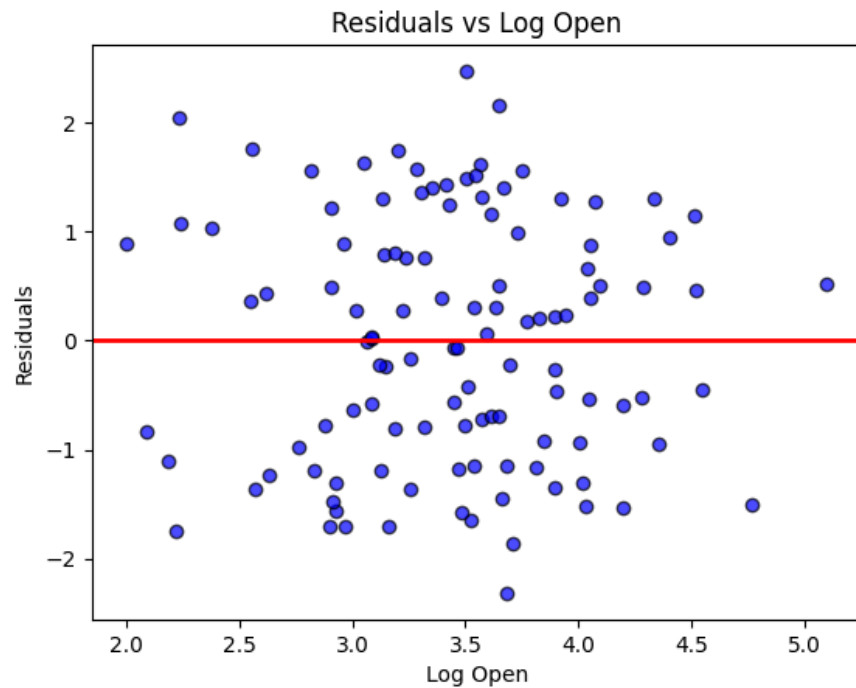
Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

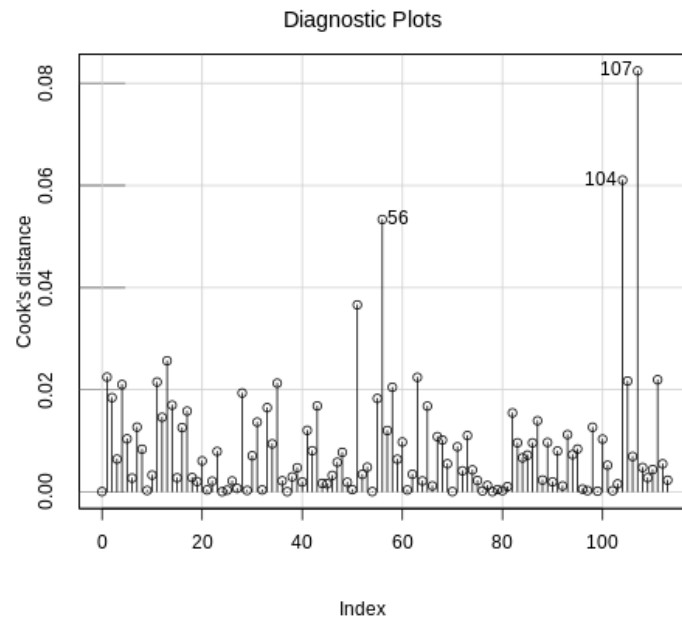




```
plt.scatter(data['lopen'], model5.resid, color='blue', edgecolor='black', alpha=0.7)
plt.axhline(y=0, color='red', linewidth=2) # Add a horizontal line at y=0
plt.ylabel("Residuals")
plt.xlabel("Log Open")
plt.title("Residuals vs Log Open")
plt.show()
```



```
%%R
library(car)
mreg.mod <- lm(lpcinc ~ lopen, data=r_df)
influenceIndexPlot(mreg.mod, id=list(n=3), vars="Cook")
```



Bootstrapping Model, testing for robustness

```
random.seed(90095)
```

```
n_boot = 500 # number of bootstraps
```

```
# initializations
```

```
boot_rmse = []
```

```
boot_b1 = []
```

```
boot_b2 = []
```

```
boot_sig2hat = [] # sigma^2 hat estimates (or std error ^2)
```

```
boot_r2 = [] # R^2 (no need to adjust)
```

```
data_span_x = pd.DataFrame({'lopen':np.linspace(1.5, 34, 200)}) # x spanning 3.5 through 34 for graphing
```

```
for iter in range(n_boot):
```

```
    # Sampling with replacement
```

```
    data_sample = data.sample(n=data.shape[0], replace=True) # `sample` method for pandas DataFrame
```

```
    data_sample_ols = smf.ols('lpcinc ~ lopen', data=data_sample).fit() # fit model
```

```
    # Append coefficients
```

```
    boot_b1.append(data_sample_ols.params[0]) # intercept
```

```
    boot_b2.append(data_sample_ols.params[1]) # slope
```

```
    # Performance measures
```

```

boot_sig2hat.append(data_sample_ols.scale)          # scale = estimated sigma^2
boot_rmse.append(np.sqrt(np.mean(data_sample_ols.resid ** 2))) # RMSE
boot_r2.append(data_sample_ols.rsquared)            # R^2

# Plot a greyed out line for this iteration
data_sample_yhat = data_sample_ols.predict(data_span_x)
plt.plot(data_span_x, data_sample_yhat, color='grey', alpha=0.1)

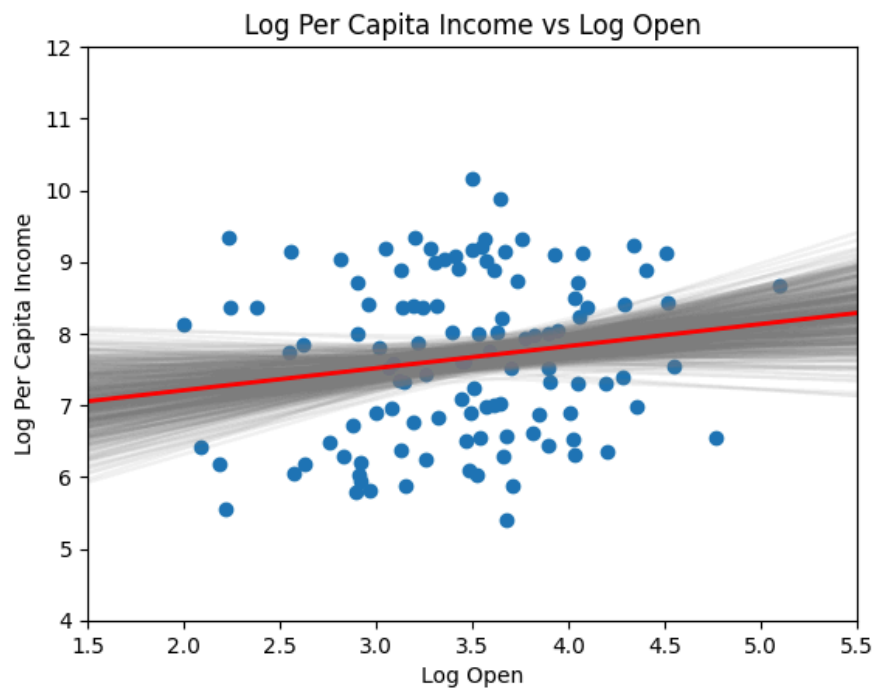
# Fit averaged model
data_ols_span_y = np.mean(boot_b1) + data_span_x * np.mean(boot_b2)

# Plotting the fitted lines
plt.scatter(data['lopen'], data['lpcinc'])
plt.plot(data_span_x, data_ols_span_y, color='r', linewidth=2)
plt.xlabel('Log Open')
plt.ylabel('Log Per Capita Income')
plt.title('Log Per Capita Income vs Log Open')

plt.xlim(1.5, 5.5) # Adjusting the range
plt.ylim(4, 12)

plt.show()

```




```

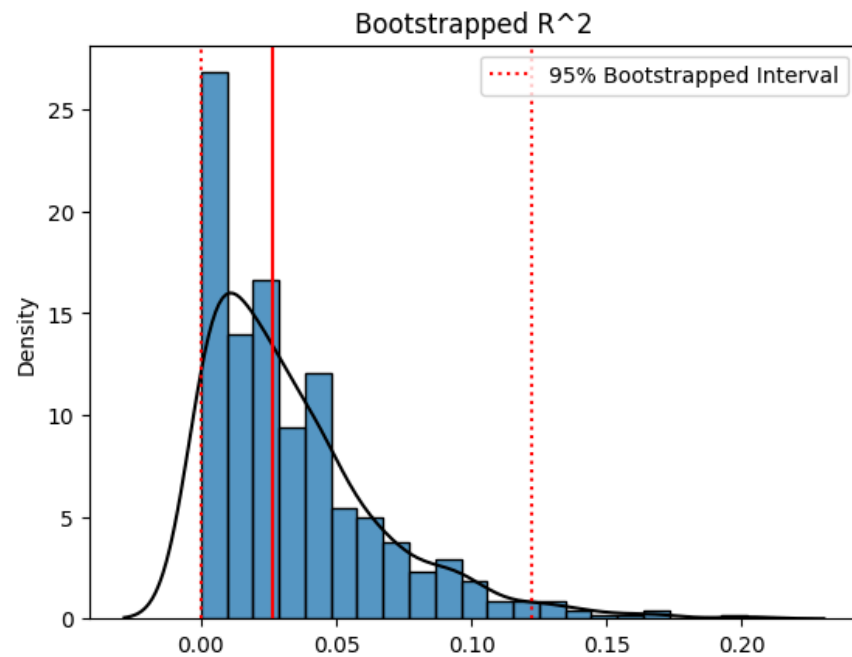
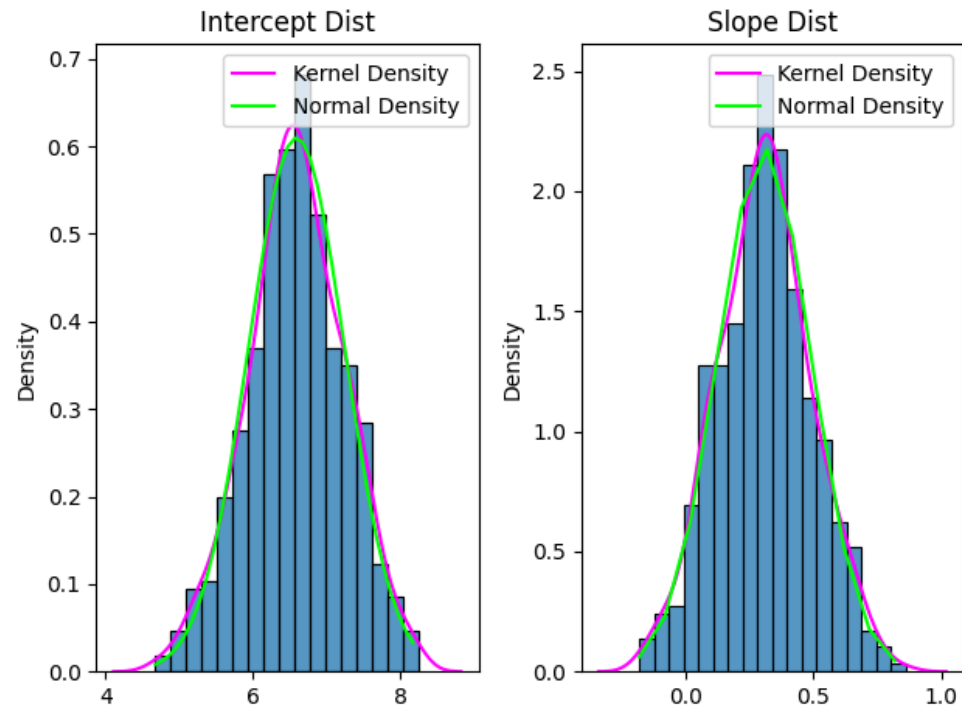
b1_range = np.arange(np.min(boot_b1), np.max(boot_b1), 0.1)
b2_range = np.arange(np.min(boot_b2), np.max(boot_b2), 0.1)

# Plotting beta estimates
fig6, ax6 = plt.subplots(1, 2)
sns.histplot(boot_b1, stat='density', kde=False, ax=ax6[0])
sns.kdeplot(boot_b1, color='fuchsia', ax=ax6[0], label='Kernel Density')
sns.lineplot(x=b1_range, y=norm.pdf(b1_range, np.mean(boot_b1), np.std(boot_b1)), color='lime', ax=ax6[0], label='Normal Density')
ax6[0].set_title('Intercept Dist')
sns.histplot(boot_b2, stat='density', kde=False, ax=ax6[1])
sns.kdeplot(boot_b2, color='fuchsia', ax=ax6[1], label='Kernel Density')
sns.lineplot(x=b2_range, y=norm.pdf(b2_range, np.mean(boot_b2), np.std(boot_b2)), color='lime', ax=ax6[1], label='Normal Density')
ax6[1].set_title('Slope Dist')
plt.tight_layout()
plt.show()

# Bootstrap for distribution of R^2 (coefficient of determination)
sns.histplot(boot_r2, stat='density', kde=False)
sns.kdeplot(boot_r2, color='black')
plt.axvline(x=np.quantile(boot_r2, q=0.025), color='red', linestyle=':', label='95% Bootstrapped Interval')
plt.axvline(x=np.quantile(boot_r2, q=0.5), color='red')
plt.axvline(x=np.quantile(boot_r2, q=0.975), color='red', linestyle=':')
plt.title('Bootstrapped R^2')
plt.legend()
plt.show()

# Bootstrap for estimated residual standard deviation
boot_stdhat = np.sqrt(boot_sig2hat)
print('--- sigma hat ---\n' +
      f'estimate:\t\t{boot_stdhat.mean():.5}\n' +
      f'std error:\t\t{boot_stdhat.std():.5}\n' +
      f'median:\t\t\t{np.quantile(boot_stdhat, q=0.5):.5}\n' +
      f'2.5 percentile:\t\t{np.quantile(boot_stdhat, q=0.025):.5}\n' +
      f'97.5 percentile:\t{np.quantile(boot_stdhat, q=0.975):.5}'
      )

```



```

--- sigma hat ---
estimate:      1.1163
std error:    0.053367

```